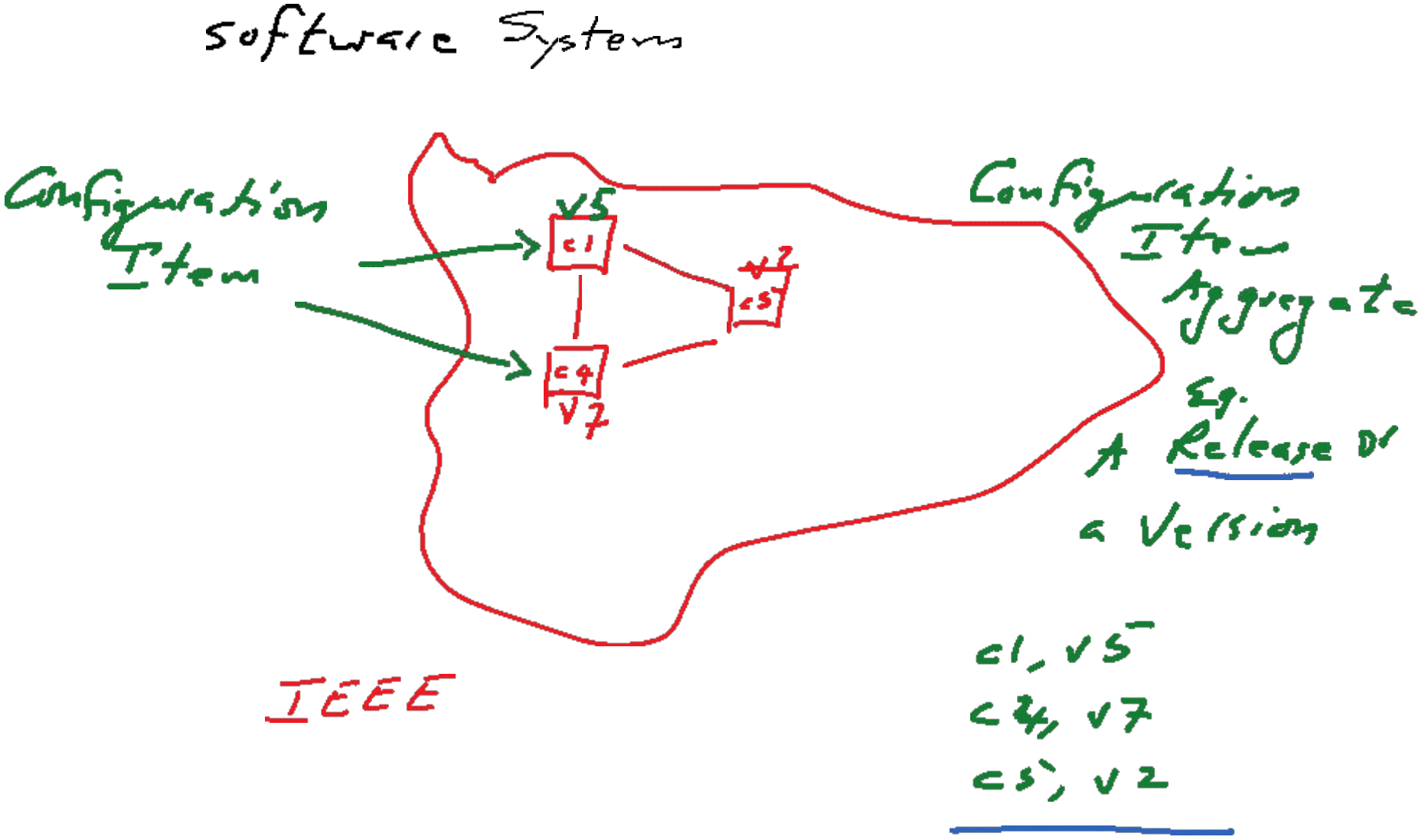
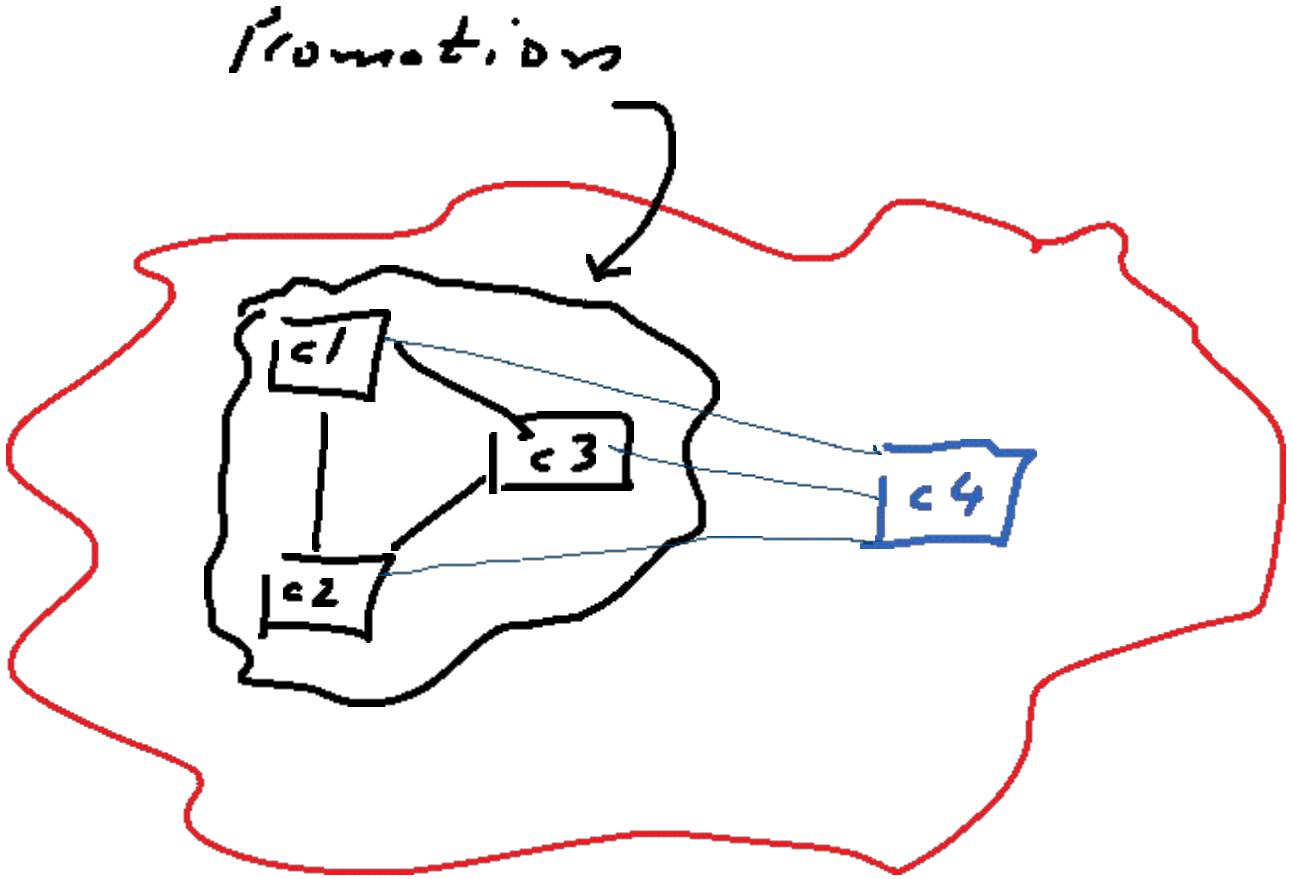
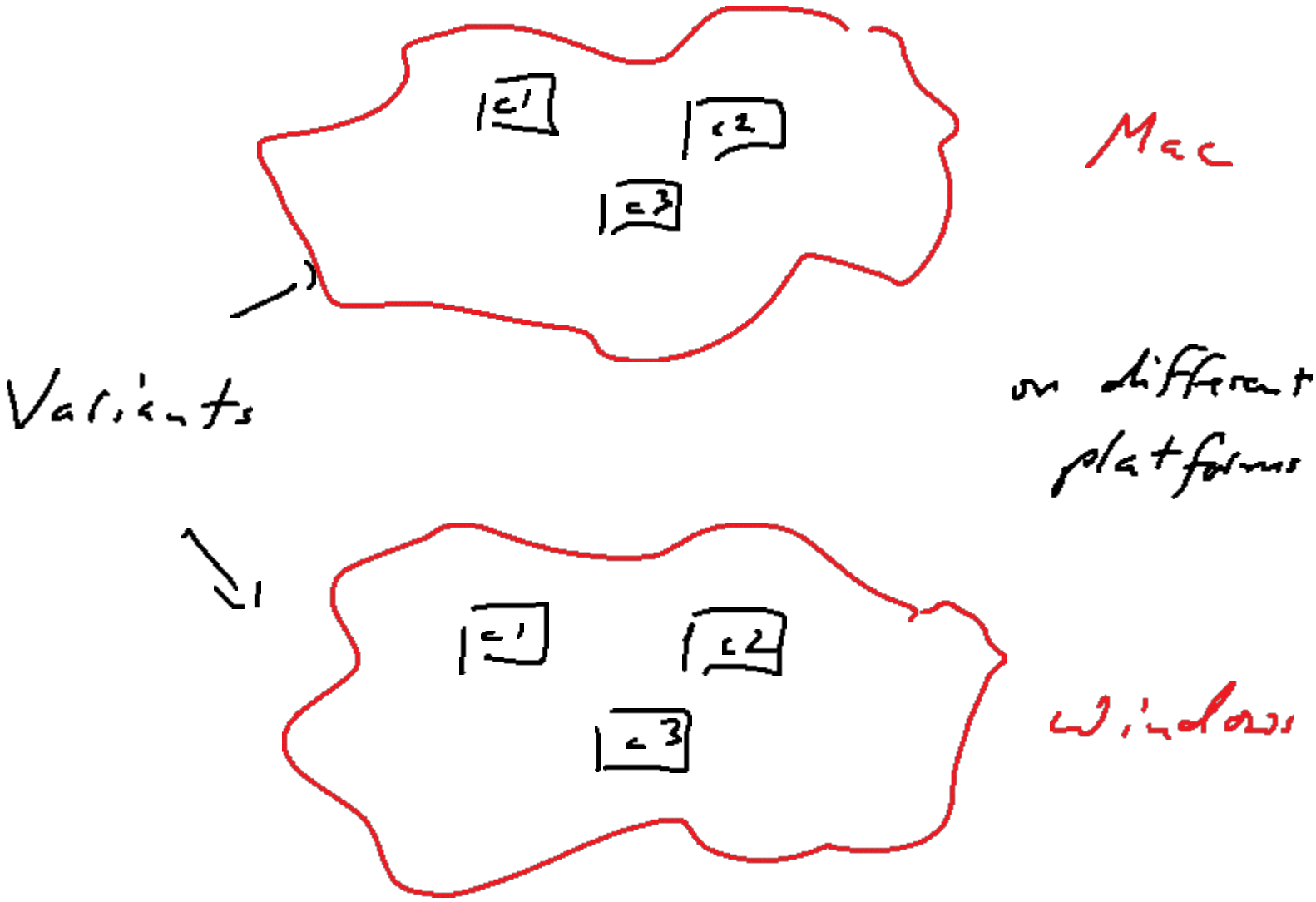
Configuration Management

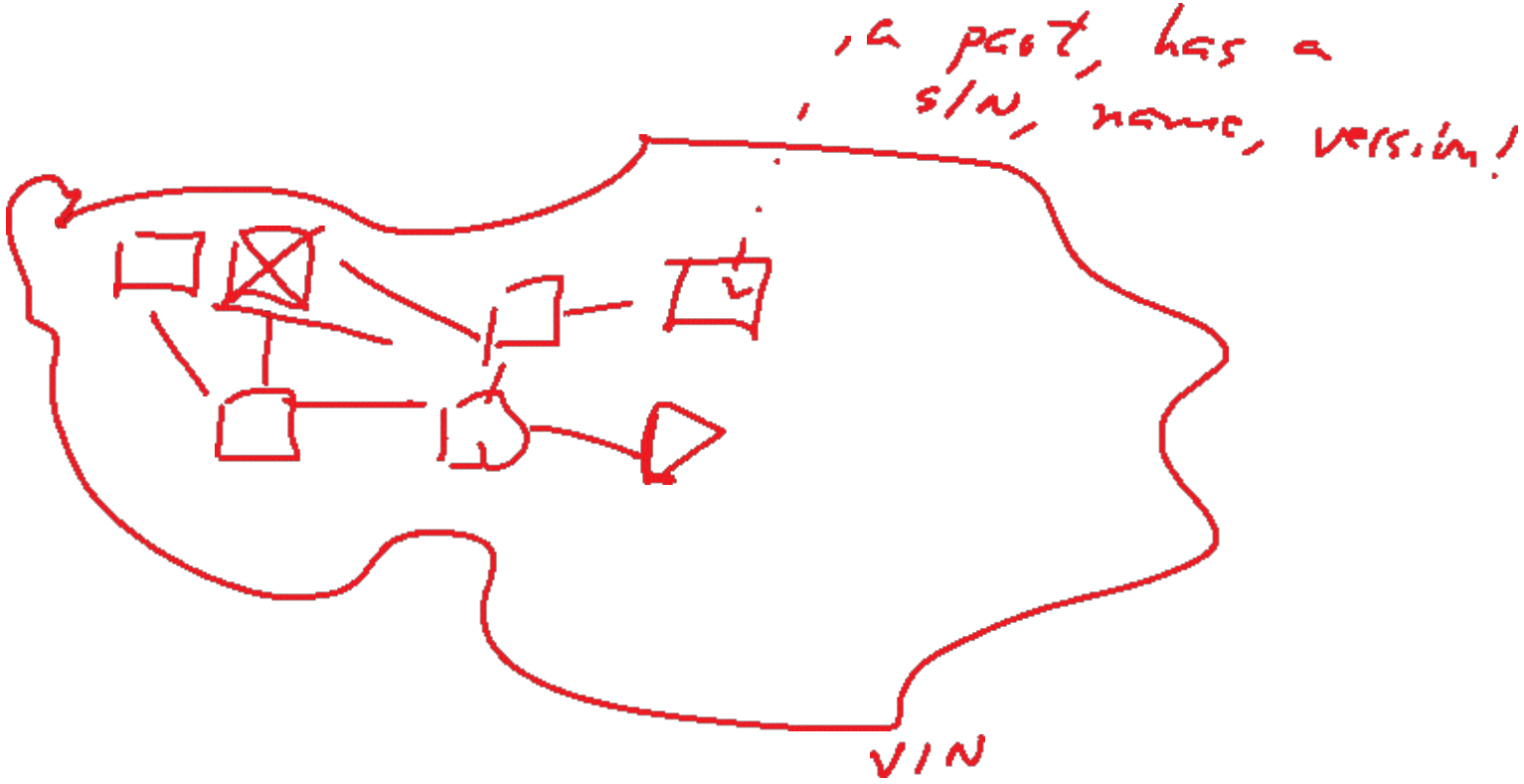

Software System
Configuration Item
v5
c1
v2
c3
c4
v7
Configuration Item
Aggregate
Eg.
A Release or
a Version
IEEE
c1, v5
c4, v7
c3, v2



c1, c2 and c3 are required before c4 can be worked on.


c1
c2
c3
Mac
Variants
on different
platforms
c1
c2
c3
Windows

1 Car



Configuration Management

- Change pervades the development process.
  - Requirements change when developers improve their understanding of the application domain.
  - The system design changes with new technology and design goals.
  - The object design changes with the identification of solution objects.
  - The implementation changes as faults are discovered and repaired.
  - These changes can affect every work product, from the system models to the source code and documentation.

Configuration Management

- Configuration management is the process of controlling and monitoring change to work products.
- Once a baseline is defined, configuration management minimizes the risks associated with changes by defining a formal approval and tracking process for changes.

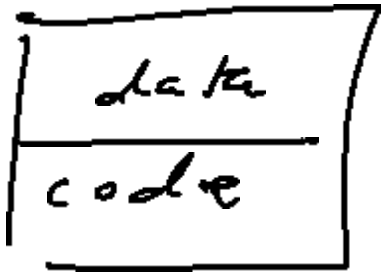
Learning Objectives

- Configuration item identification
  - The modeling of the system as a set of evolving components
- Promotion management
  - The creation of versions for other developers
- Release management
  - The creation of versions for the client and users
- Branch management
  - The management of concurrent development
- Variant management
  - The management of versions intended to coexist
- Change management
  - The handling, approval and tracking of change requests

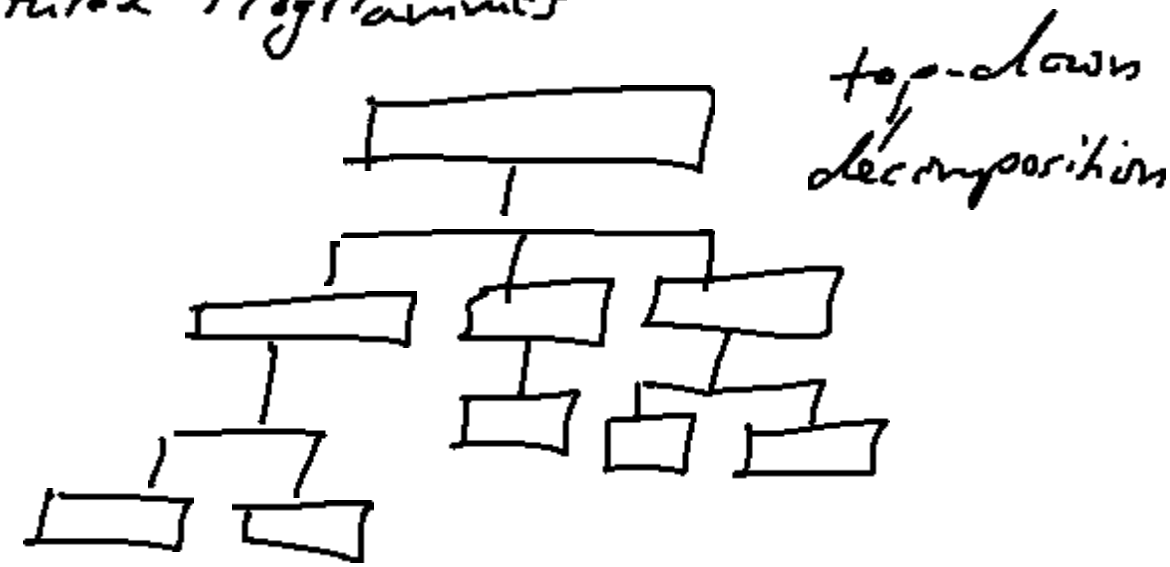
Overview

- Configuration management is the discipline of managing and controlling change in the evolution of software systems [IEEE Std. 1024-1987].
- Configuration management systems automate the identification of versions, their storage and retrieval, and supports status accounting.

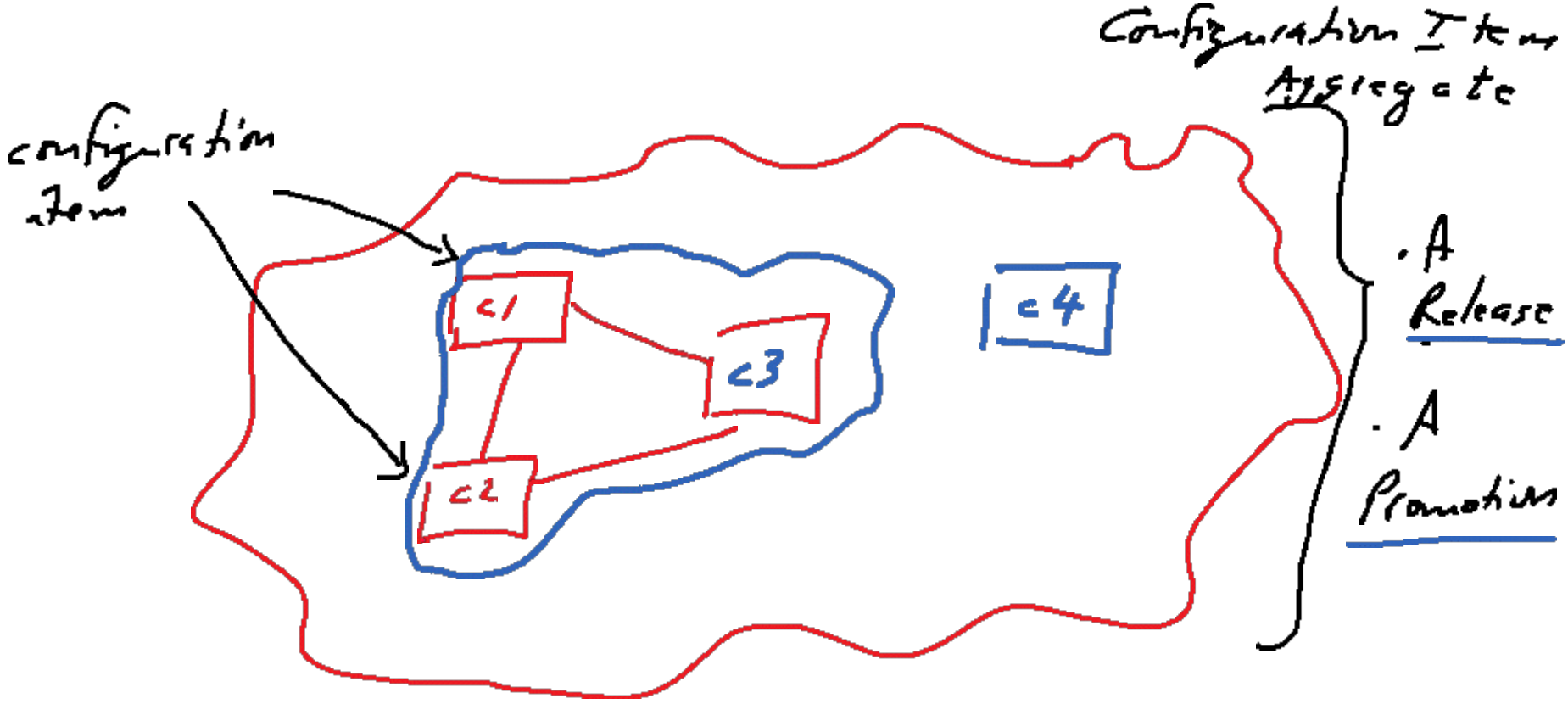
1. Lifecycle
2. OOP / Structured Programmes



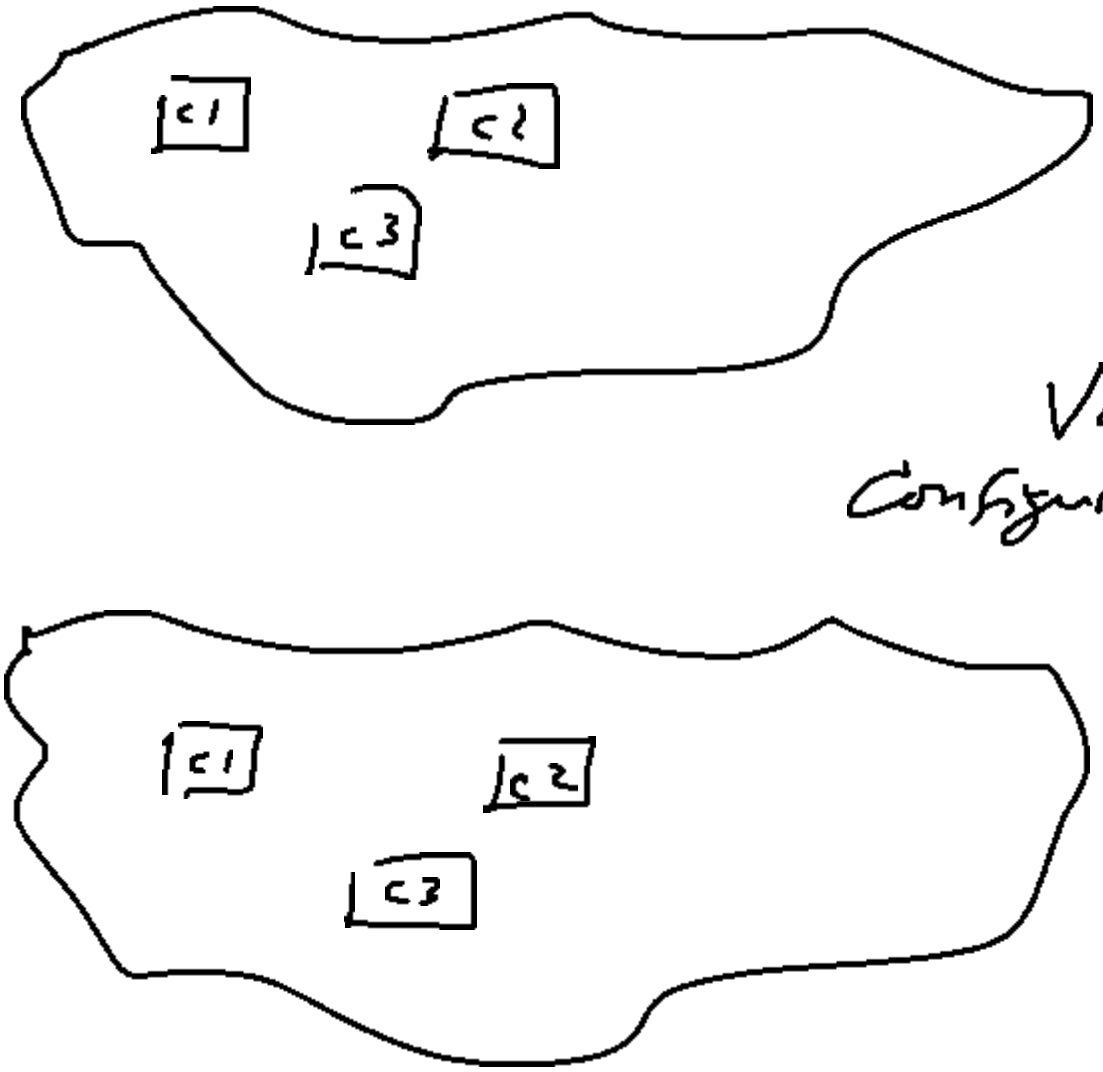
functional separation

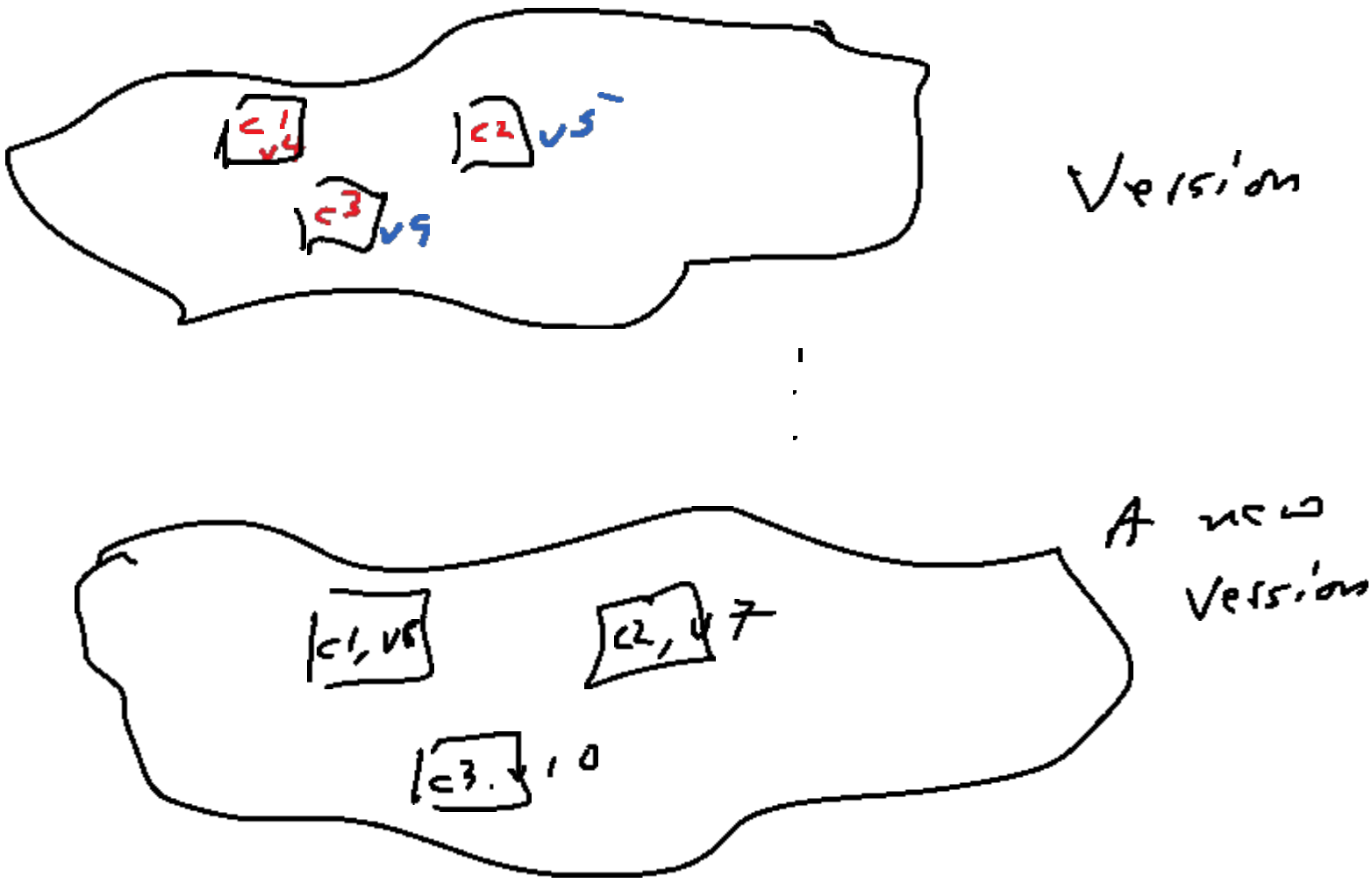


IEEE



c1, c2, c3 are a promotion (a release for developers).


c1
c2
c3
Mac
Variant
Configuration Item
Aggregate
c1
c2
c3
Windows


Version
A new Version

Overview

- Configuration management includes the following activities:
  - Identification of configuration items
  - Change control
  - Status accounting
  - Auditing
- The following are often considered part of configuration management:
  - Build management
  - Process management

Overview

- Identification of configuration items
  - Components of the system and its work products and their versions are identified and labeled uniquely.
  - Developers identify configuration items once the principal deliverables and components of the system are agreed on.
  - Developers create versions and additional configuration items as the system evolves.

Overview

- Change control
  - Changes to the system and releases to users are controlled to ensure consistency with project goals.
  - Change control can be done by the developers, by management, or by a control board, depending on the level of quality required and the rate of change.

Overview

- Status accounting
  - The status of individual components, work products, and change requests are recorded.
  - This allows developers to distinguish versions more easily and track issues related to changes.
  - This also allows management to track project status.

Overview

- Auditing
  - Versions selected for release are validated to ensure the completeness , consistency, and quality of the product.
  - Auditing is accomplished by the quality control team.

Overview

- Build management
  - Most configuration management systems enable the automatic building of the system as developers create new versions of the components.
  - The configuration management system has sufficient knowledge of the system to minimize the amount of recompilation.
  - It may also be able to combine different versions of components to build different variants of the system (e.g., for different operating system and hardware platforms).

- Traditionally, configuration management is seen as a management discipline, helping project managers with change control, status accounting, and auditing activities.
- More recently, however, configuration management has also been seen as a development support discipline, helping developers deal with the complexity associated with a large number of changes, components and variants.

Configuration Management Concepts

Terminology from IEEE Std. 1042-1987

- Configuration Item
  - A work product or a piece of software that is treated as a single entity for the purpose of configuration management.
  - A composite of configuration items is defined as a configuration management aggregate (CM aggregate)
- Change Request
  - A formal report issued by a user or a developer to request a change in a configuration item.

Configuration Management Concepts

- Version
  - Identifies the state of a configuration item or a configuration item at a well defined point in time.
  - Given a CM aggregate , a consistent set of versions of its configuration items is defined as a configuration.
  - I.e., a configuration can be thought of as a version of a CM aggregate.
- Versions that are intended to coexist are called variants

Configuration Management Concepts

- Promotion
  - A version that has been made available to other developers in the project.
- Release
  - A version that has been made available to the client or the users.
- Software Library
  - Stores versions and provides facilities to track the status of changes.
  - A repository is a library of releases.
  - A master directory is a library or promotions

Configuration Items and CM Aggregates

- Configuration Item
  - A work product or a component of a work product that is under configuration management and treated as a single entity for such purposes. For example, the device driver for a serial port in any operating system is a configuration item. This components is simple enough that it cannot be further broken down for installation.

Configuration Items and CM Aggregates

- CM Aggregate
  - A composition of configuration items.
  - E.g. , the Linux operating system is a CM aggregate that includes a process scheduler, a memory manager, many device drivers, network daemons, file systems and many other subsystems.

Versions and Configurations

- Version
  - Identifies the state of a configuration item at a well-defined point in time.
  - Successive versions of a work product differ by one or more changes, such as the correction of a fault, the addition of new functionality, or the removal or unnecessary or obsolete functionality.
  - A configuration identifies the state of a CM aggregate.

Versions and Configurations

- Baseline
  - A version of a configuration item that has been formally reviewed and agreed on by management or the client, and which can be changed only through a change request.
  - This ensures that changes are made consistently with project goals (e.g., safety and reliability) and regulations, and that they are communicated to relevant developers and users.
- Variants
  - Versions that are intended to coexist are called variants. E.g., a software system can have a Macintosh variant, a Windows variant and a Linux variant, each providing identical functionality. A system may also have a standard variant, and a professional or deluxe variant, supporting a different range of functionality.
  - Variants share a large amount of code that implements core functionality, and differences are confined to a small number of lower level subsystems.

Change Requests

- A formal step in initiating the change process.
- A user, client, or developer discovers a fault in a work product or wants a new feature.
- The author of the change request specifies the configuration item to which the request applies, its version, the problem to be solved, and a proposed solution.
- In the case of a formal change process, the costs and benefits of the change are then assessed before the change is approved or rejected. In both cases the rationale of the decision is recorded with the change request.

Promotions and Releases

- Promotion
  - A promotion is a version that is made available to other developers.
  - A promotion denotes (indicates) that a configuration item has reached a relatively stable state and can be used or reviewed by other developers. E.g., subsystems are promoted for other teams using that subsystem. Subsystems are then promoted for the quality control team to assess their quality. Later, as faults are discovered and repaired, revisions of the subsystem are promoted for reevaluation.

Promotions and Releases

- Release
  - A version that is made available to users. A release denotes that a configuration item has met the quality criteria set by the quality control team and can be used or reviewed by the users.
  - E.g., a system is released to beta testers so that they might find additional faults and assess the perceived quality of the system.
  - As faults are discovered and repaired, revisions of the subsystem are promoted for reevaluation by the quality control team and, when the quality criteria are met, released again to the users.

Repositories and Workspaces

- Configuration management systems provide a repository in which all configuration items are stored and tracked and a workspace in which the developer makes changes. Developers create new versions by checking in changes from their workspace to the repository.
- A software library, as defined in [IEEE Std. 1042-1987], provides facilities to store, label, and identify versions of the configuration items (i.e., documentation, models, and code). A software library also provides functionality to track the status of changes to the configuration items.

Repositories and Workspaces

- Workspace
  - The developer's workspace, also known as dynamic library, is used for everyday development by the developers. Change is not restricted and is controlled by the individual developer only.
- Master Directory
  - Also known as the controlled library, tracks promotions. Change needs to be approved and versions need to meet certain project criteria (e.g., only code that can be compiled without errors may be checked in) before they can be made available to the rest of the project.
- Repository
  - Also known as the static library, tracks releases. Promotions need to meet certain quality control criteria (e.g., all faults detected by regression tests must be repaired) before a promotion becomes a release.

Version Identification Schemes

- Versions are uniquely identified by developers and systems using a version identifier, also called a version number. Some exotic examples include:
  - The Ada specification went through five successive major versions, named Strawman, Woodenman, Tinman, Ironman and Steelman.
  - The version identification scheme for TeX, a typesetting system for technical texts, is based on decimals of the number π: Each time a bug is found (which is rare) and repaired, the version number of TeX is incremented to add another digit. The current version is 3.1415926.
  - In general, however, version numbers of a given configuration item can be quite large. In this case, developers and configuration management systems use version identification schemes that support automation more easily, such as sequential numbers with two or three decimals, Here is a three digit version identification scheme in Backus-Naur Form:

    <version> ::= <Configuration item name>.<major>.<minor>.<revision>

    <major>::=<nonnegative integer>

    <minor>::=<nonnegative integer>

    <revision>::=<nonnegative integer>

- Development can take place in multiple *threads*, i.e., **branches**. It is very often the case that developers work on multiple improvements concurrently.
- This implies that everyone works with the most recent version of a component they depend upon, or older stable versions of components they depend upon. This approach works fine if changes affect non-overlapping sets or components and interfaces remain backward compatible. However, when two change require modifications to the same component, a different approach is required. Concurrent branches and subsequent merging can be used to coordinate changes.
- Merging, is a nontrivial operation.

Documenting Configuration Management

- [IEEE Std. 828-1990] is a standard for writing Software Configuration Management Plans (SCMP).
- Such a plan documents all the relevant information to the configuration management activities of a specific project. The plan is generated during the planning phase, put under configuration management, and revised as necessary. The scope and length of the plan may vary according to the needs of the project.

Template

1. Introduction
    1. Purpose
    2. Scope
    3. Key terms
    4. References
2. Management
    1. Organization
    2. Responsibilities
3. Activities
4. Schedule
5. Resources
6. Plan maintenance

SCMP

Contains six types of information:

- Introduction
  - Describes the scope and audience of the document, key terms, and references
- Management
  - Describes the organization of the project (which can be a reference to the corresponding section of the Software Project Management Plan) and how the configuration management responsibilities are assigned within then organization
- Activities
  - Describes in detail the identification of configuration items, the change control process, the process for creating releases and for auditing, and the process for status accounting. Responsibilities for each of these activities are assigned to a role defined in the management section.
- Schedule
  - Denotes when the configuration activities take place and how they are coordinated. In particular it defines at which point changes can be requested and approved through the formal change process.
- Resources
  - Identifies the tools, techniques, equipment, personnel, and training necessary for accomplishing the configuration activities.
- Plain maintenance
  - Defines how the SCMP, itself under configuration management, is maintained and revised. In particular, this section deals with the person responsible for its maintenance, the frequency of updates, and the change process for updating the plan.

IEEE-SPMP
Software Project Management Plan template

- IEEE – Formerly Institute for Electrical and Electronics Engineering
- SPMP – Software Project Management Plan

IEEE Vision and Mission

- Vision
  - To advance global prosperity by fostering technological innovation, enabling members' careers and promoting community world-side.
- Mission
  - The IEEE promotes the engineering process of creating, developing, integrating, sharing and applying knowledge about electronics and information technologies and sciences for the benefit of humanity and the profession.

SPMP overview

- Project management models are summarized in the Software Project Management Plan (SPMP). This document is created before project kick-off and is updated throughout the project as tasks are completed and procedures are refined.
- The audience of the SPMP includes the management and the developers.
- The SPMP documents all issues related to client requirements (such as deliverables and acceptance criteria), the project goals, the project organization, the division of labour into tasks, and the allocation of resources and responsibilities.
- IEEE Std. 1058.1-1987 is for Software Project Management Plans. Specifies format and contents of SPMP. DOES NOT specify technique to be used or examples. Applies to all types of software projects. Applies to software projects of all sizes.

SPMP Sections

- Introduction
  - Provides background material for the rest of the document. It briefly describes the project, the client deliverables, the project milestones, project constraints.
- Project organization
  - The organizational structure of the subsystem teams and cross-functional teams. The boundaries of each team and of the management are defined and responsibilities are assigned. Communication roles, such as liaisons are also described. The software process model used in the project is described.
- Managerial process
  - Describes how management monitors project status and how it addresses unforeseen problems. Dependencies among teams and subsystems are described. Anticipated risks and contingency plans are made public. Documenting what could go wrong makes it easier for developers to report when problems actually occur. This section should be regularly updated to include newly identified risks.
- Technical process
  - Describes the technical standards that all teams are required to adopt. These cover the development methodology, the configuration management policy of documents and code, the coding guidelines, and the selection of standard off-the-shelf components. Some might be imposed by company-wide interests and others by the client.
- Work elements, schedule and budget
  - Represents the most visible product of management. This section details how work is to be carried out and by whom.

SPMP Versions

- The first version of the SPMP contains detailed plans only for the early phases of the project. Detailed plans for later phases are updated as the project progresses and important risks are better understood.
- An initial version of the SPMP should be written by the project manager during the project definition phase, and it should be coordinated with its companion document, the initial software architecture document.
- The SPMP and the software architecture are referred to as initial, because the documents should not be finalized until the beginning of the project steady state phase. Some authors only recommend a baseline of these documents only after the requirements analysis phase. Only then will all the project participants be able to understand the impact of the requirements on the work.
- The baselined work breakdown structure should be established in collaboration with the developers. The completed SPMP document should be reviewed by both management and developers to ensure that the work breakdown structure and schedule are feasible and that important risks have not been omitted.

Example SPMP document template

- **1. Introduction**
  - 1.1 Project Overview
    1.2 Project Deliverables
    1.3 Evolution of the Software Project Management Plan
    1.4 Reference Materials
    1.5 Definitions and Acronyms
- **2. Project Organization**
  - 2.1 Process Model
    2.2 Organizational Structure
    2.3 Organizational Boundaries and Interfaces
    2.4 Project Responsibilities
- **3. Managerial Process**
  - 3.1 Management Objectives and Priorities
    3.2 Assumptions, Dependencies, and Constraints
    3.3 Risk Management
    3.4 Monitoring and Controlling Mechanisms
    3.5 Staffing Plan
- **4. Technical Process**
  - 4.1 Methods, Tools, and Techniques
    4.2 Software Documentation
    4.3 Project Support Functions
- **5. Work Packages, Schedule, and Budget**
  - 5.1 Work Packages
    5.2 Dependencies
    5.3 Resource Requirements
    5.4 Budget and Resource Allocation
    5.5 Schedule
- **6. Additional Components**
- **7. Index**
- **8. Appendices**

PRINCE2 (an alternative)

http://www.prince2.com/what-is-prince2.asp

- PRINCE2 (Projects IN Controlled Environments) is a process based method for effective project management.
- It is a de facto standard used extensively by the UK government and the private sector in the UK and elsewhere.
- PRINCE2 is in the public domain, offering non-proprietorial best practice guidedance on project management. PRINCE2 is a registered trademark of OGC.

PRINCE2

- The key features are:
  - Its focus on business justification
  - A defined organizational structure for the project management team
  - Its product-based planning approach
  - Its emphasis on dividing the project into manageable and controllable stages
  - Its flexibility to be applied at a level appropriate to the project.

SPMP

1. Introduction
2. Project Organization
3. Managerial Process
4. Technical Process
5. Work Packages, Schedule and Budget
6. Additional Components
7. Index
8. Appendices

SPMP

1.0 Introduction :

This section should describe the project and the software project to be built.

1.1 Project Overview

A short summary of the project objectives, the software to be delivered, major activities, major deliverables, major milestones, required resources, and top-level schedule and budget. A description of the relationship of this project to other projects should be given, if appropriate. Generally does not cover the project's requirements.

1.2 Project Deliverables

List all of the major items to be delivered to the customer (external customer, in-house user, etc.). List the deliverables, delivery dates, delivery locations, delivery method (email, FTP, CD, etc.), and quantities necessary to satisfy the project's requirements.

1.3 Evolution of the Software Project Management Plan

Describe how this document can be expected to evolve over time. This section should list what is expected to be done to the document. This is a plan for the document revisions.

1.4 Reference Materials

A list of all the documents and other materials referenced in this document. This is like the bibliography in a published book.

1.5 Definitions and Acronyms

Definitions or references for all special terms and acronyms used within this document

SPMP

2.0 Project Organization

2.1 Process Model

Describes the following:

- The project's lifecycle model (e.g. waterfall model, spiral model, evolutionary prototyping model, etc.)
- The project's major milestones (content and dates/timing). This should include a text description of the meaning of each milestone plus a Gantt chart or other high-level description of the project's schedule.
- Major work products, including content and timing. These could be listed in a table.

Software Project Survival Guide (Might be useful to include these items):

- *Change Control Plan*
- *Change Proposals*
- *Vision statement*
- *Top 10 Risks List*
- *Software Development Plan, including project cost and schedule estimates*
- *User Interface Style Guide*
- *User Manual/Requirements Specification*
- *Quality Assurance Plan*
- *Software Architecture*
- *Software Integration Procedure*
- *Staged Delivery Plan*
- *Individual Stage Plans, including miniature milestone schedules*
- *Coding Standard*
- *Detailed Design Documents*
- *Software Construction Plans*
- *Deployment Document (Cutover Handbook)*
- *Release Checklist*
- *Release Sign-Off Form*
- *Software Project Log*
- *Software Project History Document*

SPMP

2.2 Organizational Structure

Describe the internal management structure of the project. Use organization charts, matrix diagrams, or other appropriate notations to describe the lines of authority, responsibility, and communication within the project. It is not necessary to show the organization of the entire company, only this project.

2.3 Organizational Boundaries and Interfaces

Describe the relationships between the project and each of the following organizations:

- Upper management
- Customer organization
- Subcontracting organizations (if any)
- Any organizations the project interacts with

2.4 Project Responsibilities

Identify and describe each major project function and activity, and identify the person responsible for each function and activity.

3.0 Managerial Process

Describes the management objectives, priorities, project assumptions, dependencies, constraints, risk management techniques, monitoring and controlling mechanisms, and the staffing plan.

3.1 Management objectives and Priorities

The philosophy, goals and priorities for management during the project. The following might be included:

- Relative priorities among functionality, schedule, budget and quality
- Risk management procedures
- Approach to acquiring third party software
- Approach to modifying or using existing software

3.2 Assumptions, Dependencies, and Constraints

Describe the assumptions upon which the project plans are based, the dependencies of project plans and the constraints upon the project plans. (like functionality, schedule, budget, quality, etc)

3.3 Risk Management

Describe how risks will be identified, tracked and monitored. Identify the risk manager and his/her duties. The actual risks themselves are contained in a separate document, the Risks List, that is periodically updated.

3.4 Monitoring and Controlling Mechanisms

Describe how project cost, schedule, quality and functionality will be tracked throughout the project. Consider describing the following:

- Statius and Progress report formats
- Reporting structure and frequency
- Meeting structure and frequency
- Audit mechanisms

3.5 Staffing Plan

Describe the numbers and types of personnel needed to conduct the project. Describe the required skill levels, start times, duration on the project, method of obtaining the personnel, training required and phasing out of the project personnel.

SPMP

- 4.0 Technical Process
  - This section describes the top-level technical processes used on the project including the technical methods, tools, and techniques; major software documents; and supporting activities such as configuration management and quality assurance.
  - 4.1 Methods, Tools, and Techniques
    - Describe the following:
      - *The computing system environment including hardware and operating system environment;*
      - *Software tools including design tools, source code control, time accounting, compiler or IDE, debugging aids, defect tracking, and so on*
      - *Development methodologies including requirements development practices, design methodologies and notations, programming language, coding standards, documentation standards, system integration procedure, and so on (these will not all be defined when the first draft of the project plan is created; the section should be updated as the plans become more detailed)*
      - *Quality assurance practices including methods of technical peer review, unit testing, stepping through code in a debugger, system testing, automated regression tests, etc.*

SPMP

- 4.2 Software Documentation
  - List the documents that will be developed for the project, including milestones, reviews, and signoffs for each document. If they are described in sufficient detail elsewhere, that description can simply be referenced here. The documentation list might include:
    - *Change Control Plan*
    - *Change Proposals*
    - *Vision statement*
    - *Top 10 Risks List*
    - *Software Development Plan, including project cost and schedule estimates*
    - *User Interface Style Guide*
    - *User Manual/Requirements Specification*
    - *Quality Assurance Plan*
    - *Software Architecture*
    - *Software Integration Procedure*
    - *Staged Delivery Plan*
    - *Individual Stage Plans, including miniature milestone schedules*
    - *Coding Standard*
    - *Detailed Design Documents*
    - *Software Construction Plans*
    - *Deployment Document (Cutover Handbook)*
    - *Release Checklist*
    - *Release Sign-Off Form*
    - *Software Project Log*
    - *Software Project History Document*

SPMP

- 4.3 Project Support Functions
  - Describe or give references to other documents that describe the plans for functions that support the software development effort, including configuration management, quality assurance, and end user documentation. Whether included in this document or described elsewhere, the plans for these supporting functions should be developed in as much detail as the plans for the software itself are. (This includes responsibilities, resource requirements, schedule, budget, etc.)

SPMP

- 5.0 Work Packages, Schedule, and Budget
  - 5.1 Work Packages
    - Describe the work package (i.e., task or collection of tasks) that must be completed to complete the software. Identify each work package with a unique number, and provide a diagram showing the breakdown of work packages into sub-packages.
  - 5.2 Dependencies
    - Describe the dependencies both among work packages and between the project's work packages and external events
  - 5.3 Resource Requirements
    - Describe the resources required for the project week by week or month by month, depending on the length of the project. Describe the numbers and type of personnel that are used, number of computers and software used, office facilities used, training needed, budget used, and so on over the whole course of the project.
  - 5.4 Budget and Resource Allocation
    - Describe the portion of the project's budget allocated to different functions (e.g., engineering, quality assurance, documentation, management). Describe how the budget will be expended over the course of the project
  - 5.5 Schedule
    - Describe the schedule for the various project functions, activities, and tasks, taking into account the various dependencies and required milestone dates and deliverable dates. You can express schedules either as absolute calendar dates or as times relative to key project milestones (e.g., "requirements signed off + 60 days"). [The schedule is updated frequently and may be more appropriately maintained as a separate document, in which case include a link here.]

SPMP

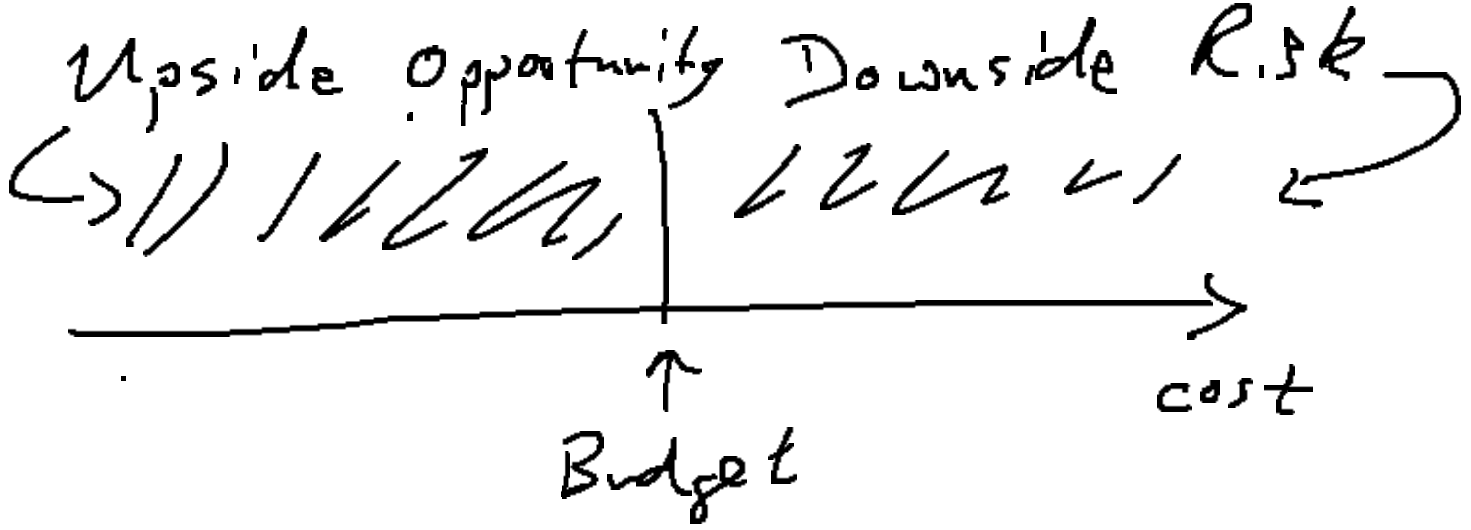
- 6.0 Additional Components
  - Include additional components needed to manage your specific project. Possibilities include subcontractor management plans, security plans, training plans, hardware procurement plans, facilities plans, installation plans, cutover plans, and software maintenance plans
- 7.0 Index
  - The index is optional according to the IEEE standard. If the document is made available in electronic form, readers can search for terms electronically.
- 8.0 Appendices
  - Include supporting detail that would be too distracting to include in the main body of the document.

Risk Management

Project (another view…)

- “An endeavour in which human, material and financial resources are organized in a novel way, to undertake a unique scope of work of given specification, within constraints of cost and time, so as to achieve unitary, beneficial change, through the delivery of quantified and qualitative objectives.” (Turner, 1992)

- The definition of a project highlights:
  - the once-off, change inducing nature of projects,
  - the need to organize a variety of resources under significant constraints,
  - and the central role of objectives in project definition.
- It also suggests an inherent uncertainty which requires attention as part of effective project management processes.


Upside Opportunity
Downside Risk
cost
Budget

Project uncertainty

- The roots of project uncertainty are associated with six basic questions which need to be addressed:

Who	Who are the stakeholders ultimately involved in the project?	Stakeholders
Why	What do the stakeholders want to achieve?	Motives
What	What is it the stakeholders are interested in?	Design
Which way	How is it to be done?	Activities
Wherewithal	What resources are required?	Resources
When	When does it have to be done?	Schedule

Project Risk

- Broad definition
  - “the implications of the existence of significant uncertainty about the level of project performance achievable”
- A source of risk is any factor that can affect project performance, and risk arises when this effect is both uncertain and significant in its impact on project performance.
- It follows that the definition of project objectives and performance criteria has a fundamental influence on the level of project risk.
  - Setting tight cost or time targets makes a project more cost or time risky by definition, since achievement of targets is more uncertain if targets are ‘tight’.
  - Conversely, setting slack time (activity float) or quality requirements implies low time or quality risk. However, inappropriate targets are themselves a source of risk, and a failure to acknowledge the need for a minimum level or performance against certain criteria automatically generates risk on those dimensions.

Project Risk

- It has been argued that it is important to set clear objectives and performance criteria which reflect the requirements of various parties , including stakeholders that are not always recognized as players (regulatory authorities, for example).
- The different project objectives held by interested parties and any interdependencies between different objectives need to be appreciated. Strategies for managing risk cannot be divorced from strategies for managing project objectives.
- Whatever the underlying performance objectives, the focus on project success and uncertainty about achieving it leads to risk being defined in terms of a ‘threat to success’.
- Suppose success for a project is measured solely in terms of realized cost relative to some target or commitment. Then risk might be defined in terms of the threat to success posed by a given plan in terms of the size of possible cost overruns and their likelihood.
- From this perspective it is a natural step to regard risk management as essentially about removing or reducing the possibility of underperformance. This is a limited appreciation of project risk since there is also the positive side of uncertainty which may present opportunities rather than threats.

Role of Risk Management

- To improve project performance by way of systematic identification, appraisal and management of project related risk.
- A focus on reducing threats or adverse outcomes (‘downside’ risk). An aim of improving performance implies a wider perspective which seeks to exploit opportunities or favourable possibilities, which has been called the ‘upside potential’.
- To realize, in practical terms, the advantages of this wide perspective, it is essential to see project risk management as an important extension of conventional project planning, which has the potential to influence project design and baseline plans on a routine basis, occassionally influencing very basic issues like the nature of the *who*, and the *why*.

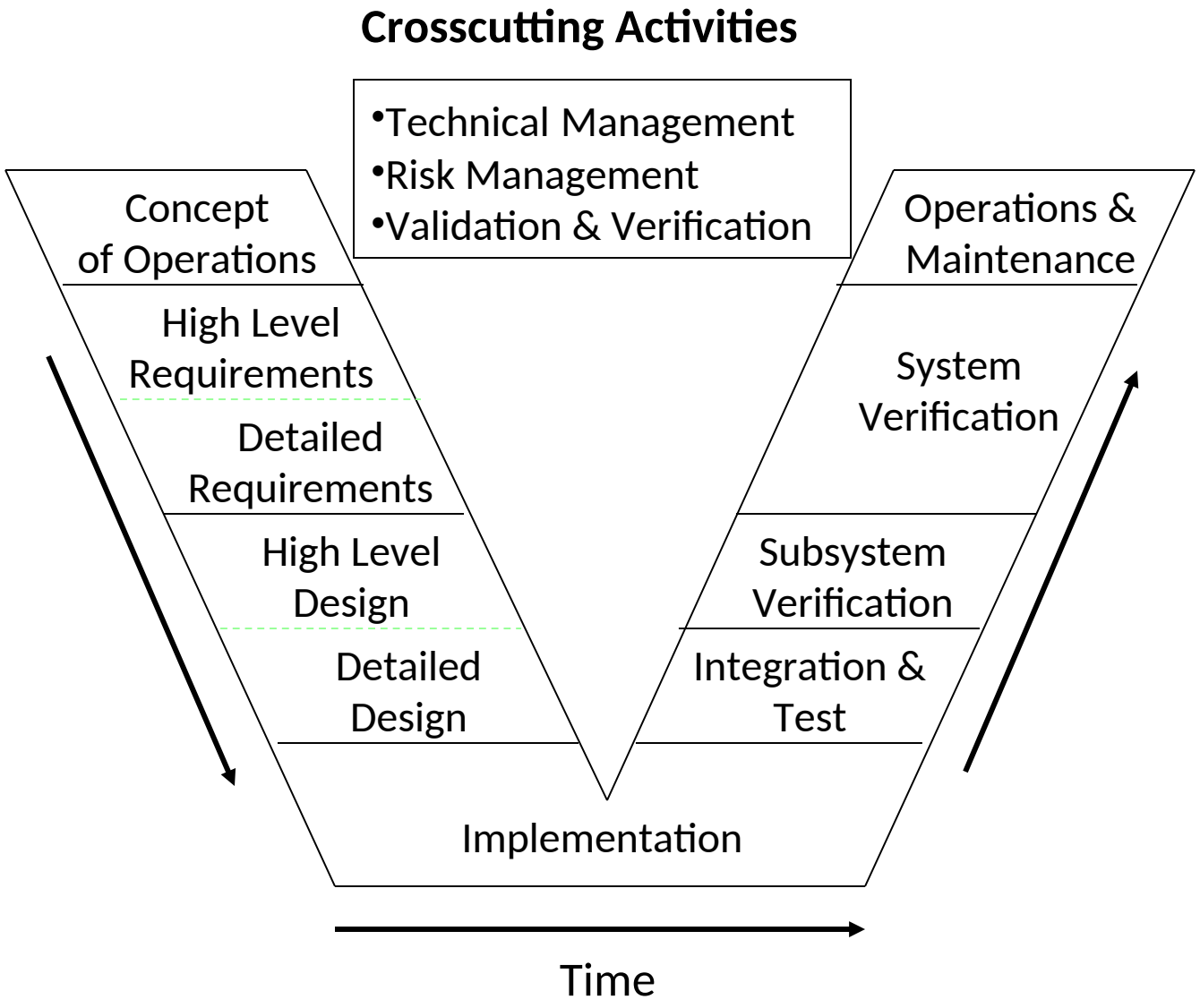
Role of Risk Management

- A ‘baseline plan’ is a target scenario, a portrayal of how we would like a project to go. This plan provides a basis for project preparation , execution, and control. Underlying the base plan is a design which may be worth identifying as a ‘baseline design’, if possible changes are anticipated.
- Control involves responding to threats and opportunities, and revising performance targets where appropriate. Contingency plans are a second level of plan, a portrayal of how we would like to respond to threats or opportunities associated with a base plan.
- Risk management is usually associated with the evaluation and development of contingency plans supporting the base plan, but effective risk management will also be instrumental in the development of project based plans. Really effective risk management will influence design as well, and it may influence motives and stakeholders.

Role of Risk Management

- Planning and risk management in this sense is proactive, rather than reactive. Where uncertainty presents potential future threats or opportunities, proactive planning and risk management seeks to modify the future incidence and quality of threats and opportunities and their possible impact on project performance.

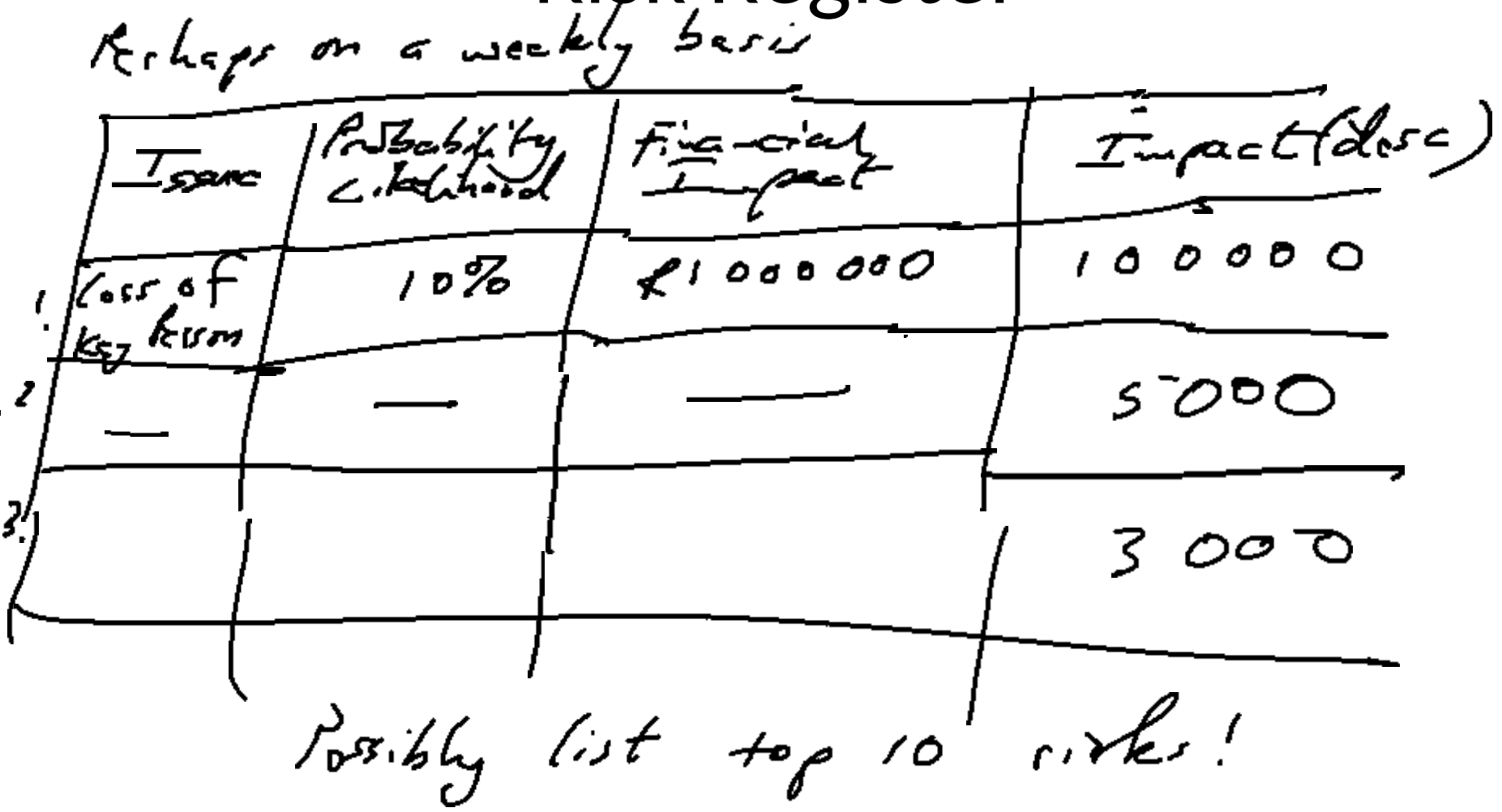
Systems Engineering Life Cycle



Representative Project Risks

Risk	Examples
Threats to the project	Inadequate funding Facilities not ready
Technology	Performance problems Interface problems
People	Low skill levels Fear of change
Management	Requirements changes Inadequate resources

Risk Register



Representative Project Risks

- **Threats to the project** - may be caused by multiple year funding that does not occur. Another example is that the control center is not constructed in time, or utilities are not installed either at the control center or for the field equipment.

- **Technology threats** – These occur when the technology does not exist to satisfy a requirement. For example, the speed of the communications system is not adequate to guarantee that a message will be displayed on a VMS within a predetermined period of time.

- **People threats** – this could include fear of the project, inability to comprehend the system operation, union problems, etc.

- **Management problems** – This does not necessarily refer to upper management, but rather project management issues. For example, not controlling the rate at which requirements change, or not assigning adequate numbers of people to do the work. In both cases, both the client and the contractor may be to blame.

Sources of Risk – One view

- Technology
- People
- Physical environment
- Political environment
- Contract issues

Sources of risk

- Technology – may be an enabler, but it is also a source of risk. If a piece of technology is made a critical element of a system, and that technology does not do what it is expected to do, it can have a significant impact on the plans for implementing the system.
- People are also a source of risk. This could arise from mistakes or the loss of personnel and can have a significant impact on the success of a project.
- The physical environment is a source of risk. It is not usually a major one however. There are obvious risks, such as fire, flood, wind etc that can cause natural disasters and threaten a project or its hardware and facilities.
- The political environment can be risky as well. Political environment risks can also include new legislation or regulations that invalidate key decisions taken earlier on in a project. This can lead to a need for unplanned reworking of parts of a project.
- Contract issues can lead to difficulties. For instance a strike by workers is a contract dispute that can affect the execution of a project.
- Increased Risks = Increased Costs

Boehm's List

- Personnel shortfalls, addressed by:
  - Obtain quality people
  - Team building
- Controlling dynamic requirements:
  - A high change threshold
- Controlling externally provided project components
  - Reference checking
  - Preaward audits
- Real-time performance risk
  - Benchmarking
  - Prototyping
  - Technical analysis
- Contingent development methods
  - Incremental development
  - Prototyping and requirements scrubbing
  - Mission analysis
  - Project de-escalation
- Unrealistic estimates

‘Ten Golden Rules’

- Make Risk Management part of your project
- Identify risks early in your project
- Communicate about risks
- Consider both threats and opportunities
- Clarify ownership issues
- Prioritize risks
- Analyze risks
- Plan and implement risk responses
- Register project risks
- Track risks and associated tasks

Risk Item	Probability of Occurrence	Financial Impact
~~	low	~~
	high	
	medium	

Risk Identification

- Risk Identification – Writing down all the worries and concerns of those concerns of those connected with the project, then continually pressing all team members to think of even more concerns.
- Risk Retirement (Mitigation) – the process whereby risks are reduced or even eliminated
  - Avoiding
  - Conquering

Important risks (e.g. with top-10 priorities) have to be managed through well-defined plans.

- Why, what, when, who, where, how, how much

Important techniques

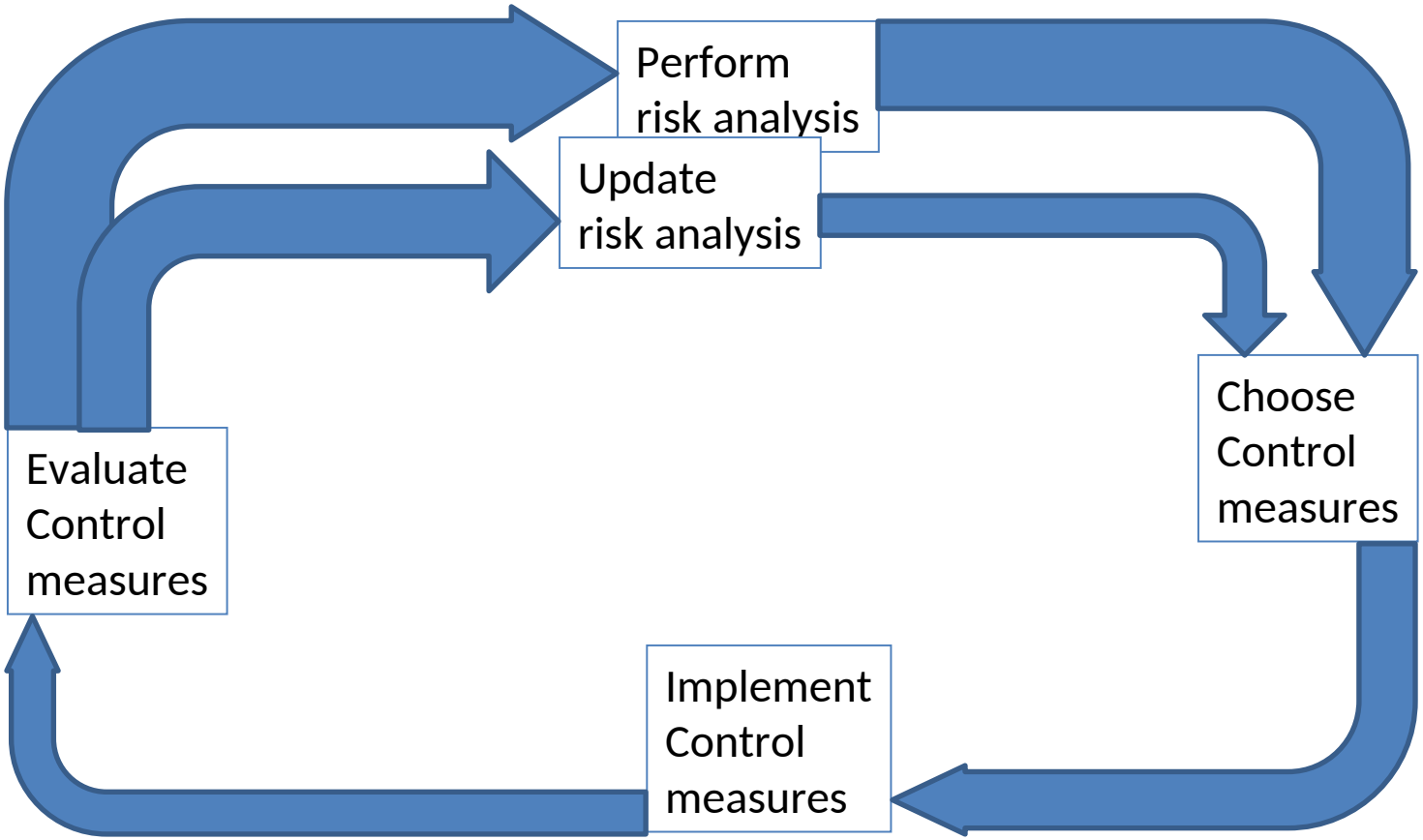
- Buying information
- Risk avoidance
- Risk transfer
- Risk reduction

Plans have to be integrated into main project plan

Risk-Management Techniques

- Personnel shortfalls
  - Top talents, job matching, team building, training
- Unrealistic schedule and budget
  - Detailed estimation, incremental development, reuse
- Developing wrong functionality
  - Prototyping, analysis, user participation
- Requirements changes
  - High change threshold, information hiding, incremental development
- External jobs
  - Benchmarking, inspection, compatibility analysis, preaward audit

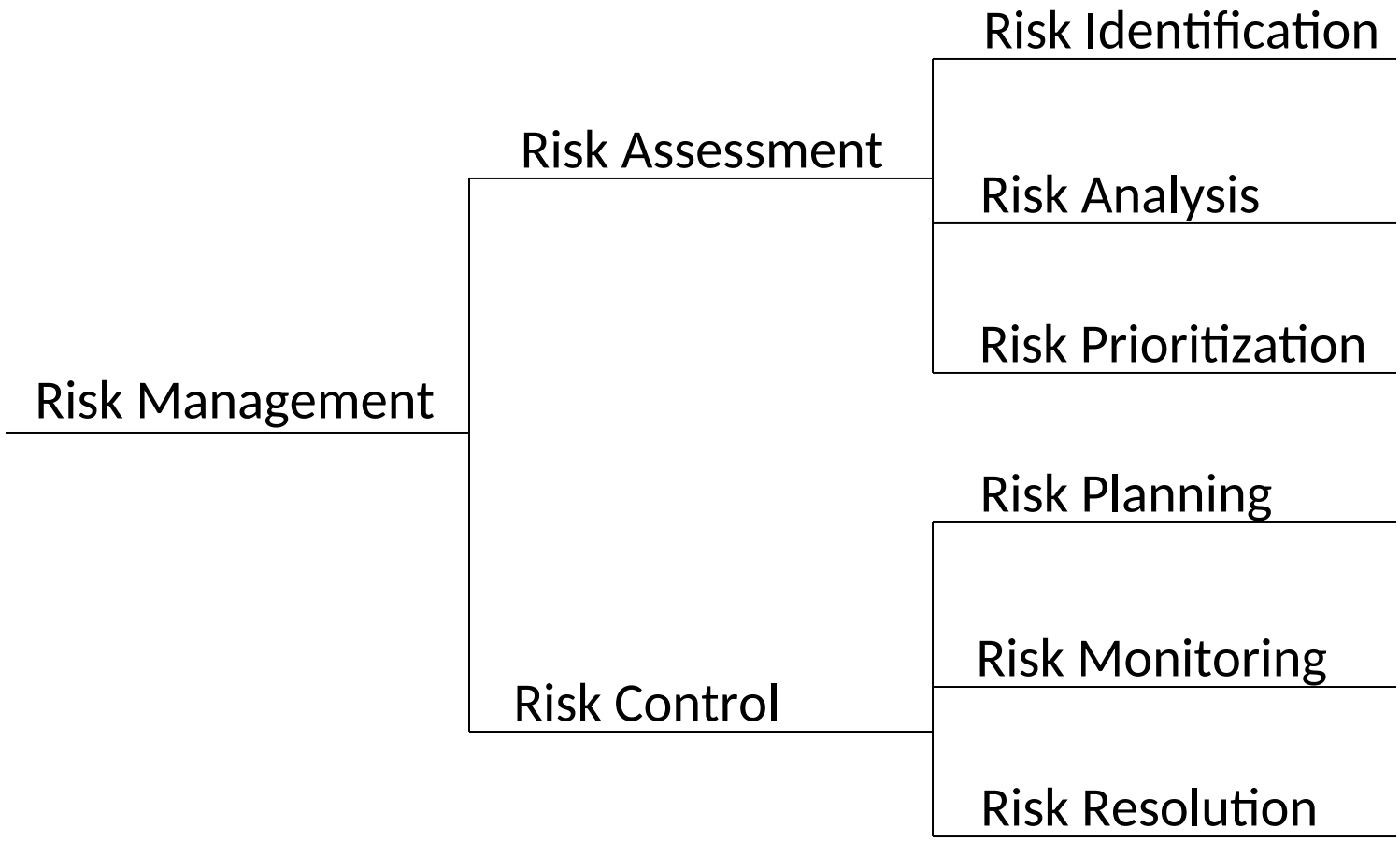
Risk Management



Top software risk items

- Personnel shortfalls
- Unrealistic schedules and budgets
- Developing the wrong functions and properties
- Developing the wrong user interface
- Continuing stream of requirements changes
- Shortfalls in externally furnished components
- Shortfalls in externally performed tasks
- Real-time performance shortfalls

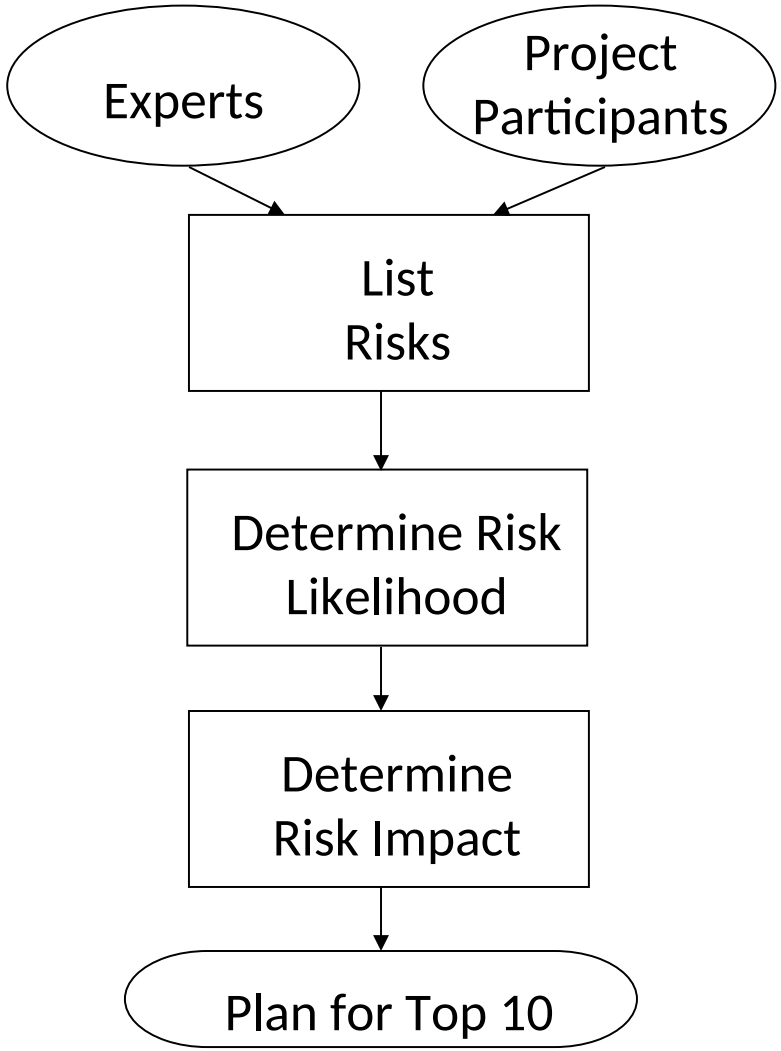
Elements of Risk Management



Risk Assessment Considerations

- Likelihood of occurrence (probability) - How likely is the risk to occur. This can be expressed as a probability (0-1 or a percentage).
- Potential impact – One way to measure impact is to calculate how much money it will cost if the risk occurs. This is sometimes difficult or impossible to do however.
- Mitigation Cost – What will it cost to mitigate the risk? If possible the cost of mitigating the risk should be compared to the expected cost of the risk.

A Sample Qualitative Process



NB: This is only a sample. There are many other approaches that can be considered.

Risk Assessment Output (Quantitative)

		Likelihood of Occurrence		
		High	Medium	Low
Impact	High	A	B	C
	Medium	D	E	F
	Low	G	H	I

Risk Assessment Output

This chart describes a qualitative approach to identifying and prioritizing potential risks. The key thing about this chart is that once you decide what cell the risk falls into, you can relatively easily decide which ones you want to focus on.

Any risk that falls into the upper left-hand corner group of cells numbered A, B and D are important. They have a high likelihood of occurrence and at least a medium impact or a high impact and at least a medium likelihood of occurrence. If you have time, you might consider the risks that fall into cells, E, C and F, in that order.

The matrix is completed by a consensus exercise in which it is essential that both supplier and purchaser participate. First, everyone in the room brainstorms potential risks. Each sector (public and private) is likely to be surprised by the risks that the other sector is worrying about.

Ask the participants to vote on the likelihood of occurrence. Assign risks to rows based on the results of the vote. This is typically done by giving each participant five paste-on dots which they stick onto the list of risks. The risks with the most dots win.

Repeat the voting process for impact. Again, categorize the results based on the number of votes received.

More than one risk is placed in each cell.

This exercise provides a side benefit of encouraging communications between the supplier (systems integrator) and the purchaser (the local agency).

Risk Management Exercise

Identify a major project in
your domain of expertise
and create a risk
assessment matrix for it

Risks Can Be Assessed Quantitatively

- Probability
- Severity (cost) of impact
- Mitigation cost

Expected Risk Cost =

Probability x Cost of impact

- If you can come up with numeric values for each of these elements, you can do a quantitative assessment of the risks on your project.

- Let us say you had a risk that, if it occurred, would cost the project an additional R100,000. At first glance, that would seem like a major risk. However, supposed you assessed the likelihood of that risk occurring as being one in 100. Using the formula:

- Expected Risk Cost = $100,000 x (1/100)
- Or
- Expected Risk Cost = $100,000/100 = $1,000

- So the calculation of “expected risk costs” yields a value of $1,000.

- That means you shouldn’t invest more than $1,000 to mitigate the risk.

- It is unlikely that this process provides more reliable answers than the qualitative process, and it does not create the team-building benefits that the qualitative process offers. However, understanding the form of the calculation helps put things in perspective.

- Since it is unlikely that you will be able to accurately estimate either the probability or the cost of impact of a risk, this approach is of little value. It is mentioned here to demonstrate the importance of estimating both the impact of the risk (Cost of impact) and its likelihood of occurrence (probability) as part of the risk management exercise.

Top 10 Generic Risks (1 through 5)

- Personnel shortfalls
- Unrealistic schedules and budgets
- Functions not right
- User interface not right
- Gold-plating

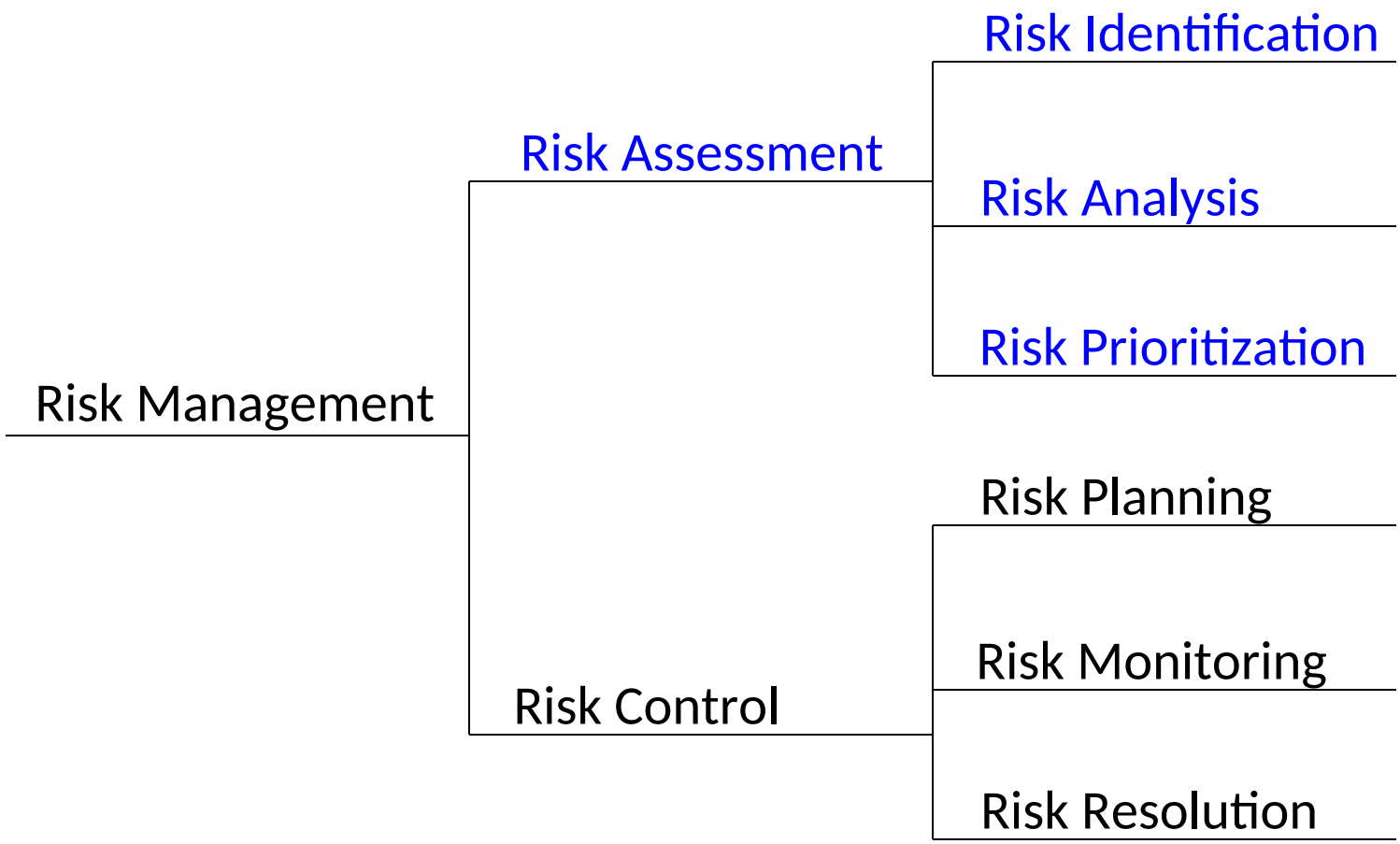
Source: Kemerer

Top 10 Risks (6 through 10)

- Requirements changes (scope creep)
- Component shortcomings
- External dependencies (subcontractors, government furnished equipment, etc.)
- Real-time performance shortfalls
- Unrealistic technical requirements

Source: Kemerer

Risk control



Risk Planning

- Ensuring that risk cannot occur
- Reducing probability of occurrence
- Planning procedures to address risks if they do occur
- Monitoring plan

Risk Planning

- Decide which risks to address. Some will have so little likelihood of occurrence or so little impact that you can ignore them. If a risk is unlikely, it doesn't mean it won't occur, but it is reasonable to bypass it in planning a risk mitigation strategy.

- Focus on those risks that could occur with enough potential impact that you should protect against them. There's no one way to do this; you'll have to decide how you want to do it and a lot of it will depend on how willing you are to take chances. A lot depends on how much you can afford to spend on "insurance."

- This plan is essential to the management of risks. It will also encourage collaborative relations between the contractor and the purchaser, since both have to agree with the provisions of the plan.

- Regular monitoring is essential if risks are to be contained before they cause significant problems.

Risk Monitoring

- Monitoring should be based on metrics.
- Risk management plan should:
  - Identify symptoms of 10 top risks
  - Define frequency with which they are checked
- When risk is identified the plan includes risk resolution measures

Risk Resolution

- Defined by plan for each risk
- Must be achievable
- Should project anticipated results

Let’s Practice – Risk #1

Risk:
Contractors worry that agencies will not review
design documentation (requirements, specifications, etc.)

When the final system does not meet their expectations
they are asked to revise the final product – at great cost!

Questions:
How would you avoid the risk?

What is your mitigation strategy?

Risk #1 – Some Answers

Planning/control/monitoring:

(1) Include submissions and review times in schedule
(2) At monthly review meetings discuss schedule and impacts of review slippages. (If any)
(3) Agree that contractor will NEVER proceed with work that depends on completed review

Resolution:

(1) Plan should include identification of compensation to contractor for schedule slippage caused by agency.

Let’s Practice – Risk #2

Risk:
Agencies worry that contractors will not assign adequate personnel to keep the project on schedule.

Questions:
How would you avoid the risk?

What is your mitigation strategy?

Risk #2 – Some Answers

Planning/control/monitoring:

(1) Review schedule at monthly meetings
(2) Include metrics at review meetings
(3) Insist that contractor identify actions to be taken when slippage occurs.

Resolution:

(1) Identify functions to be eliminated in the event of schedule slippage.
(2) Identify 2nd contractor to perform some of the work. i.e. reduce contractor's scope.
(3) Agree to extend the schedule

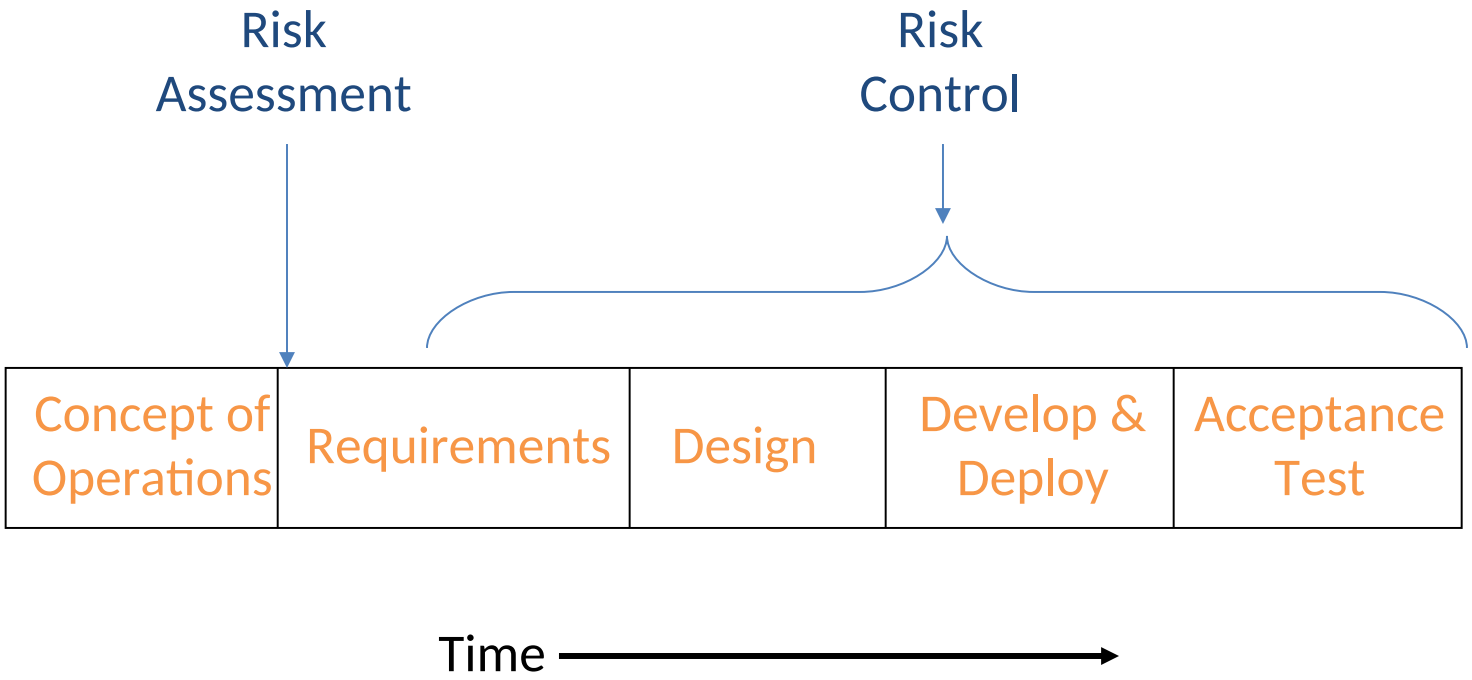
Document Your Risk Mitigation Strategy

- Identify top risks
- Quantify expected impact
- Develop plan for monitoring risks
- Define a resolution strategy for each risk

Bottom Line

- Know your risks
- Understand their impact
- Plan for their mitigation
- Monitor
- Execute the plan
- Get stakeholder buy-in

Risk Management and the Timeline



Learning Objectives

- List the most common risks
- List the elements of risk management
- Describe the process of risk assessment

- Have a look at the following paper for some detail on software development failures in the UK.
- cis.uws.ac.uk/research/journal/V9/V9N3/failure.doc

Quality

Objectives

- Explain the nature of quality
- Explain the purpose of a quality review and how it is conducted
- Describe the Capability Maturity Model (CMM)
- Describe the mechanics of auditing a project
- NB: please note relevance of material on configuration management

Quality management activities

- Quality planning
  - Select applicable procedures and standards for a particular project and modify these as required.
- Quality assurance
  - Establish organisational procedures and standards for quality.
- Quality control
  - Ensure that procedures and standards are followed by the software development team.
- Quality management should be separate from project management to ensure independence.

Quality

- The totality of features and characteristics of a product or service that bear on its ability to satisfy stated or implied needs.
- I.e. a product should meet its specifications.
  - The software product should deliver the required functionality (functional requirements) with the required quality attributes (non–functional) requirements

Quality with respect to software systems

- This is usually a problem for software systems
  - There is often tension between between customer quality requirements (viz. efficiency, reliability, etc) and developer quality requirements (viz. maintainability, reusability, etc).
  - It is difficult to specify some quality requirements in an unambiguous way.
  - Software specifications are usually incomplete and often inconsistent.
- It is not possible to wait for everything to be known before putting in place measures to ensure quality in our product
- Quality attributes are frequently conflicting and increase development costs, so there is a need for weighting and balancing of concerns.

Monitoring quality

- Quality is monitored by reviewing and testing what has been produced.
- A quality review is a review of the quality of some part of the work.
  - Earlier in the project there will have been quality reviews of the requirements, the estimates and the plan.
  - Once in execution there will be reviews of the work in progress and testing of finished products.
- A prime example is the design review, a necessary step in all projects that have a design phase.

Design Review

- A design review is a formal, documented, comprehensive and systematic examination of a design to evaluate the design requirements and the capability of the design to meet these requirements and to identify and propose solutions.

Quality attributes for a product

- For quality software it is necessary to have a number of attributes beyond meeting functionality requirements:
  - Efficiency: Software should not be wasteful with system resources such as memory, CPU time, disk space etc. Response times should be appropriate.
  - Usability: ease of use, i.e. software must be usable by the users for which it was designed.
  - Dependability: reliability, availability, security, safety, etc.
  - Maintainability: software must evolve to meet changing needs. It costs more to maintain software than to develop it.

Other quality attributes

- Resilience
- Robustness

- Understandability
- Testability
- Adaptability
- Modularity
- Simplicity

- Portability
- Reusability
- Learnability

Software metric

- Any type of measurement which relates to a software system, process or related documentation
  - Lines of code in a program, the Fog index, number of person-days required to develop a component.
- Allow the software and the software process to be quantified.
- May be used to predict product attributes or to control the software process.
- Product metrics can be used for general predictions or to identify anomalous components.
- Assumptions
  - A software property can be measured.
  - A relationship exists between what we can measure and what we want to know. We can only measure internal attributes but are often more interested in external software attributes.

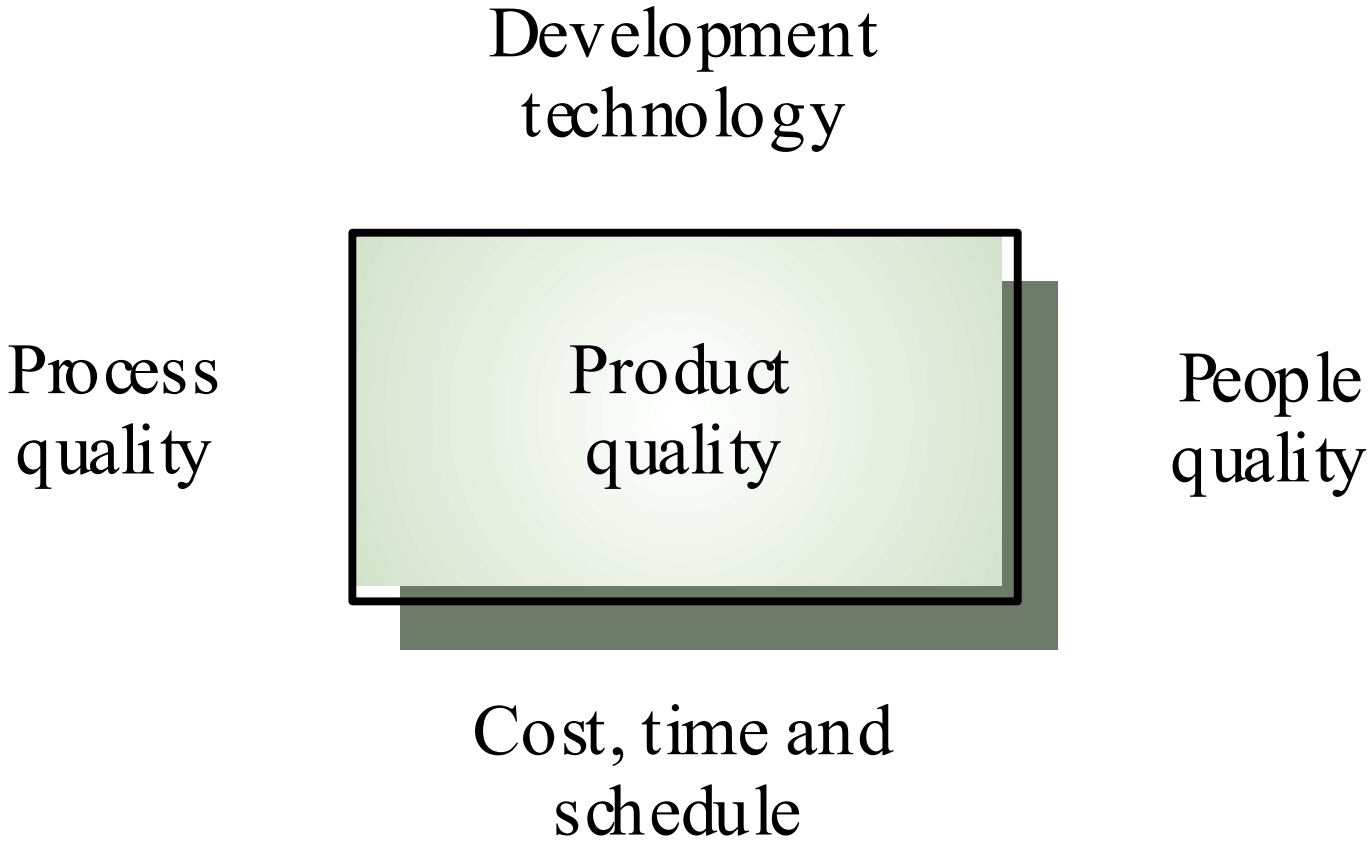
Aspects of dependability

- Reliability: The probability of system operation without failure over a specified time in a certain environment for a particular purpose.
- Availability: The probability that a system will be operational and able to deliver a requested service at a particular point in time.
- Safety: Can the system operate without danger of causing human injury or death and without damage to the system's environment. (Very significant with embedded systems).
- Security: A system's ability to protect itself from accidental or deliberate external attack.

Dependability and critical systems

- For critical systems, it is usually the case that the most important system property is the dependability of the system
- Types of critical systems:
  - ***Safety–critical system*** – a system whose failure may result in injury, loss of life or major environment damage
    - e.g. an insulin delivery system
  - ***Mission-critical system*** – a system whose failure may result in the failure of some goal-directed activity
    - e.g. a navigational system for a space aircraft
  - ***Business-critical system*** – a system whose failure may result in the failure of the business using the system
    - e.g. a customer account system in a bank

Product Quality Factors



Product Quality Metrics

- Intrinsic product quality is usually measured by the number of “bugs” (functional defects) in the software or by how long the software can run before encountering a “crash”. Two examples are:
  - Mean time to failure:- time between failures
  - Defect density:- defects relative to the software size (lines of code, function points, etc). I.e. number of defects over the opportunities for error (OFE) during a specific time frame.

Product Quality Metrics

- Customer Satisfaction Metrics:
  - Customer satisfaction:- Often measured via a five point scale
    - Very satisfied
    - Satisfied
    - Neutral
    - Dissatisfied
    - Very dissatisfied
  - Satisfaction with the overall quality of a product and its specific aspects/dimensions is usually obtained through various methods of customer surveys. For instance, the specific parameters of customer satisfaction monitored by IBM include: capability, functionality, usability, performance, reliability, installability, maintainability, documentation/information, service, and overall)

Product Quality Metrics

- Customer problems: measures the problems that customers encounter when using the product.
  - For the defect rate metric, the numerator is the number of valid defects. However, from the customer's viewpoint, all problems they encounter while using the software product, not just valid defects are problems with the software.
  - Problems that are not valid defects may be usability problems, unclear documentation, etc.

- Process quality
  - A good process is usually required to produce a good product
  - For manufactured goods, process is the principal quality determinant
  - For design-based activity (like software development), other factors are also involved especially the capabilities of the designers
  - For **large projects** with ‘average’ capabilities, the development process determines product quality
- People quality
  - For **small projects**, the capabilities of the developers is the main determinant
  - Corollary: you need lower quality people (and higher quality process) in larger projects?
  - Project Size x People Quality = Constant ?
- Development technology
  - Is particularly significant for **small projects**
- Budget and schedule
  - In **all projects**, if an unrealistic schedule is imposed then product quality will suffer

ISO Approach

- Quality Planning: identifying which quality standards are relevant to the project and determining how to satisfy them.
- Quality Assurance: evaluating overall project performance on a regular basis to provide confidence that the project will satisfy the relevant quality standards.
- Quality Control: monitoring specific project results to determine if they comply with relevant quality standards and identifying ways to eliminate causes of unsatisfactory performance.

- There are a number of standards and approaches to quality management. Among the more prominent are the following:
  - ISO standards, i,.e. ISO 9000 and 10000 series of standards and guidelines.
  - Proprietary processes such as those recommended by Deming, Juran, Crosby, and others.
  - Non-proprietary approaches such as Total Quality Management (TQM), Continuous Improvement, and others.

Quality Planning

- Identifying which quality standards are relevant to the project and determining how to satisfy them.
- A key facilitating process during project planning and should be performed regularly and in parallel with other project planning processes.

Quality assurance and standards

- Standards are the key to effective quality management.
- They may be international, national, organizational or project standards.
- **Product standards** define characteristics that all components should exhibit e.g. a common programming style.
- **Process standards** define how the software process should be enacted.

Importance of standards

- Encapsulation of best practice- avoids repetition of past mistakes.
- They are a framework for quality assurance processes - they involve checking compliance to standards.
- They provide continuity - new staff can understand the organisation by understanding the standards that are used.

Product and process standards

Product standards	Process standards
Design review form	Design review conduct
Requirements document structure	Submission of documents to CM
Method header format	Version release process
Java programming style	Project plan approval process
Project plan format	Change control process
Change request form	Test recording process

Problems with standards

- They may not be seen as relevant and up-to-date by software engineers.
- They often involve too much bureaucratic form filling.
- If they are unsupported by software tools, tedious manual work is often involved to maintain the documentation associated with the standards.

Standards development

- Involve practitioners in development. Engineers should understand the rationale underlying a standard.
- Review standards and their usage regularly. Standards can quickly become outdated and this reduces their credibility amongst practitioners.
- Detailed standards should have associated tool support. Excessive clerical work is the most significant complaint against standards.

Process and product quality

- The quality of a developed product is influenced by the quality of the production process.
- This is important in software development as some product quality attributes are hard to assess.
- However, there is a very complex and poorly understood relationship between software processes and product quality.

Process-based quality

- There is a straightforward link between process and product in manufactured goods.
- More complex for software because:
  - The application of individual skills and experience is particularly imporant in software development;
  - External factors such as the novelty of an application or the need for an accelerated development schedule may impair product quality.
- Care must be taken not to impose inappropriate process standards - these could reduce rather than improve the product quality.

Practical process quality

- Define process standards such as how reviews should be conducted, configuration management, etc.
- Monitor the development process to ensure that standards are being followed.
- Report on the process to project management and software procurer.
- Don’t use inappropriate practices simply because standards have been established.

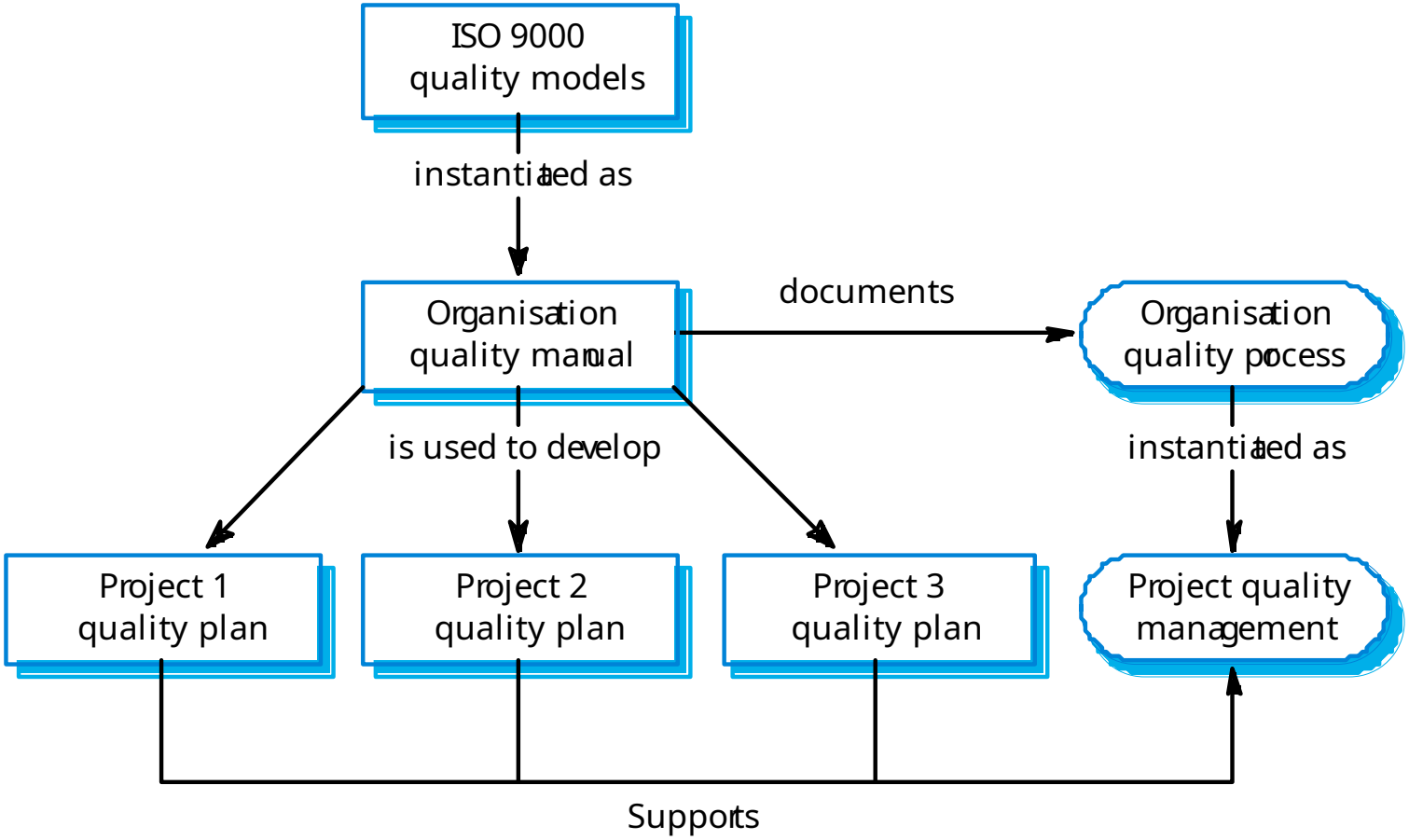
ISO 9000

- An international set of standards for quality management.
- Applicable to a range of organisations from manufacturing to service industries.
- ISO 9001 applicable to organisations which design, develop and maintain products.
- ISO 9001 is a generic model of the quality process that must be instantiated for each organisation using the standard.

ISO 9001

Management responsibility	**Quality system**
Control of non-conforming products	Design control
Handling, storage, packaging and delivery	Purchasing
Purchaser-supplied products	Product identification and traceability
Process control	Inspection and testing
Inspection and test equipment	Inspection and test status
Contract review	Corrective action
Document control	Quality records
Internal quality audits	Training
Servicing	Statistical techniques

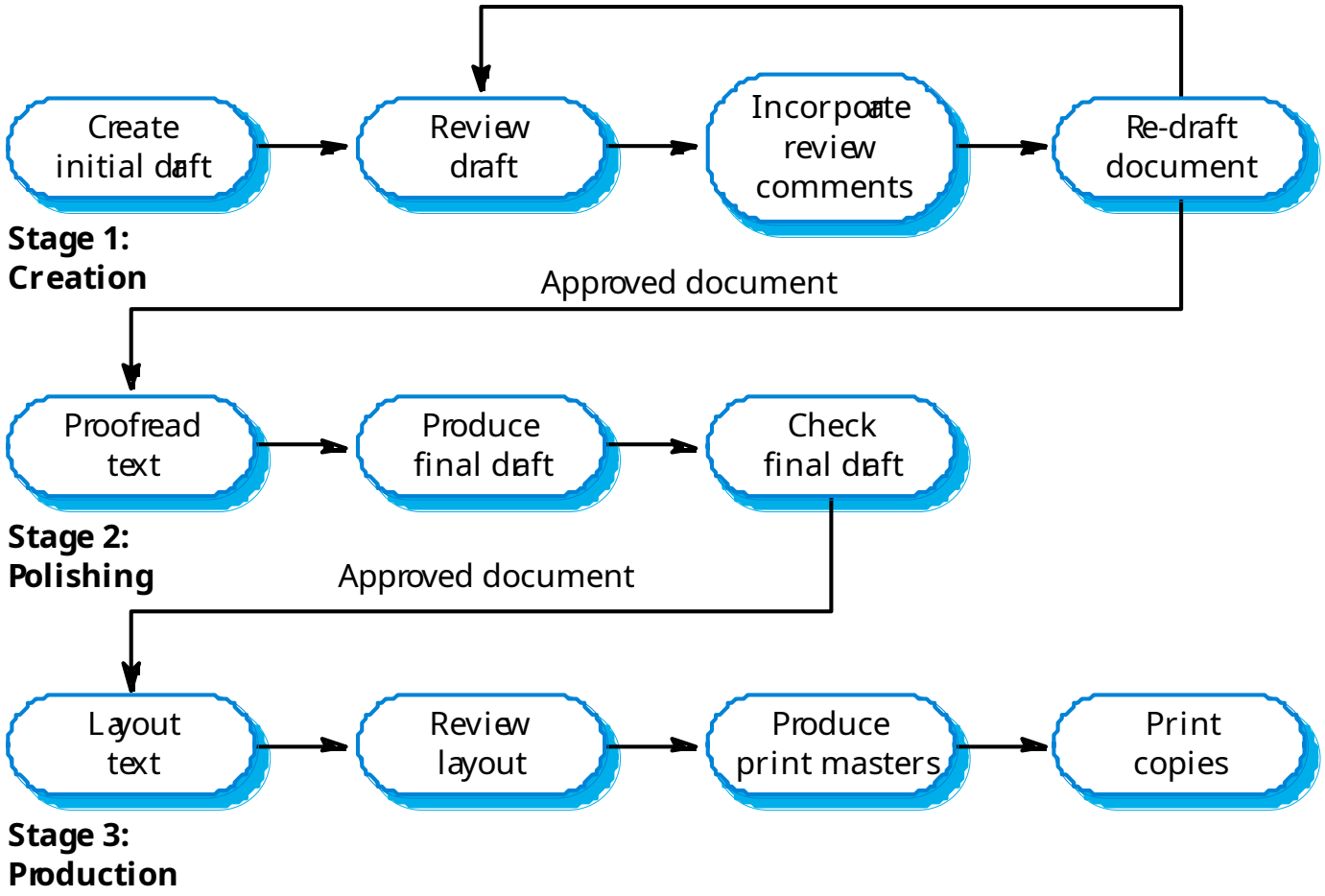
ISO 9000 and quality management



Documentation standards

- Particularly important - documents are the tangible manifestation of the software.
- Documentation process standards
  - Concerned with how documents should be developed, validated and maintained.
- Document standards
  - Concerned with document contents, structure, and appearance.
- Document interchange standards
  - Concerned with the compatibility of electronic documents.

Documentation process



Document interchange standards

- Interchange standards allow electronic documents to be exchanged, mailed, etc.
- Documents are produced using different systems and on different computers. Even when standard tools are used, standards are needed to define conventions for their use e.g. use of style sheets and macros.
- Need for archiving. The lifetime of word processing systems may be much less than the lifetime of the software being documented. An archiving standard may be defined to ensure that the document can be accessed in future.

Quality plans

- Quality plan structure
  - Product introduction;
  - Product plans;
  - Process descriptions;
  - Quality goals;
  - Risks and risk management.
- Quality plans should be short, succinct documents
  - If they are too long, no-one will read them.

Software quality attributes

Safety	Understandability	Portability
Security	Testability	Usability
Reliability	Adaptability	Reusability
Resilience	Modularity	Efficiency
Robustness	Complexity	Learnability

Quality control

- This involves checking the software development process to ensure that procedures and standards are being followed.
- There are two approaches to quality control
  - Quality reviews;
  - Automated software assessment and software measurement.

Quality reviews

- This is the principal method of validating the quality of a process or of a product.
- A group examines part or all of a process or system and its documentation to find potential problems.
- There are different types of review with different objectives
  - Inspections for defect removal (product);
  - Reviews for progress assessment (product and process);
  - Quality reviews (product and standards).

Types of review

Review type	Principal purpose
Design or program inspections	To detect detailed errors in the requirements, design or code. A checklist of possible errors should drive the review.
Progress reviews	To provide information for management about the overall progress of the project. This is both a process and a product review and is concerned with costs, plans and schedules.
Quality reviews	To carry out a technical analysis of product components or documentation to find mismatches between the specification and the component design, code or documentation and to ensure that defined quality standards have been followed.

Quality reviews

- A group of people carefully examine part or all of a software system and its associated documentation.
- Code, designs, specifications, test plans, standards, etc. can all be reviewed.
- Software or documents may be 'signed off' at a review which signifies that progress to the next development stage has been approved by management.

Review functions

- Quality function - they are part of the general quality management process.
- Project management function - they provide information for project managers.
- Training and communication function - product knowledge is passed between development team members.

Quality reviews

- The objective is the discovery of system defects and inconsistencies.
- Any documents produced in the process may be reviewed.
- Review teams should be relatively small and reviews should be fairly short.
- Records should always be maintained of quality reviews.

Review results

- Comments made during the review should be classified
  - No action. No change to the software or documentation is required;
  - Refer for repair. Designer or programmer should correct an identified fault;
  - Reconsider overall design. The problem identified in the review impacts other parts of the design. Some overall judgement must be made about the most cost-effective way of solving the problem;
- Requirements and specification errors may have to be referred to the client.

Quality Assurance

- Inputs
  - Quality management plan
  - Results of quality control measurements
  - Operational Definitions
- Tools and Techniques
  - Quality planning tools and techniques
  - Quality audits
- Outputs
  - Quality improvement

Quality Control

- Inputs
  - Work results
  - Quality management plan
  - Operational definitions
  - Checklists
- Tools and Techniques
  - Inspection
  - Control charts
  - Pareto diagrams
  - Statistical sampling
  - Flowcharting
  - Trend analysis
- Outputs
  - Quality improvement
  - Acceptance decisions
  - Rework
  - Completed checklists
  - Process adjustments

Capability Maturity Model (CMM)

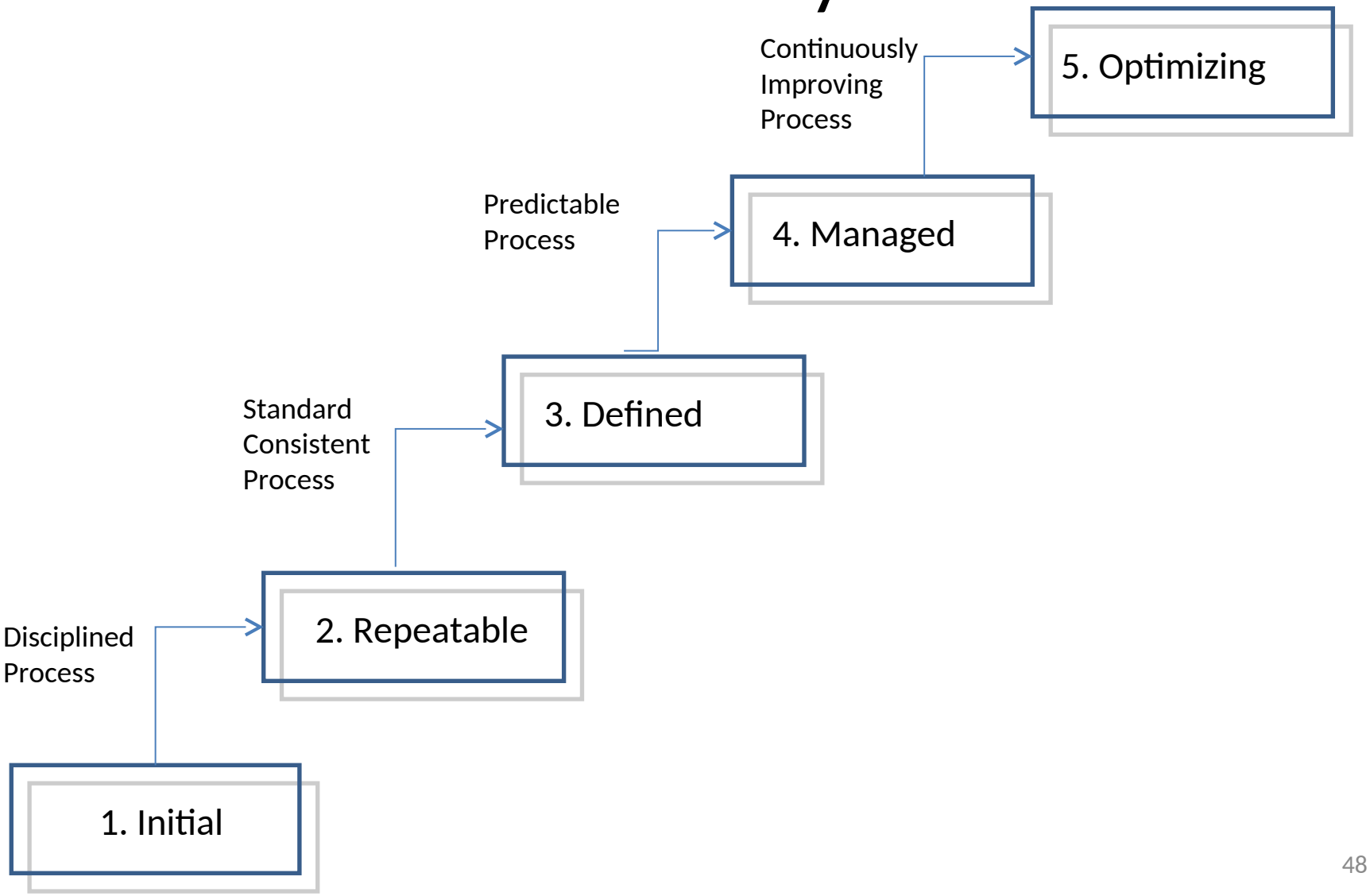
Historical: has since evolved into CMM-I

SEI Software Engineering Institute

Characterizing the Maturity of Software Life Cycle Models

- CMM uses the following five levels to characterize the maturity of an organization in terms of its commitment to *processes*.
  1. Initial
  2. Repeatable
  3. Defined
  4. Managed
  5. Optimized

Five Levels of Software Process Maturity



Key Process Areas (KPAs)

- A set of key process areas (KPA) has been defined by the Software Engineering Institute (SEI).
- To achieve a specific level of maturity, the organization must demonstrate that it addresses all the key process areas defined for that level.

Mapping Process Maturity Level and Key Process Areas

Maturity Level	Key Process Area
1. Initial	Not applicable
2. Repeatable	Focus: Establish basic project management controls. •***Requirements management***: Requirements are baselined in a project agreement and maintained during a project. •***Project planning and tracking***: A software project management plan is established at the beginning of the project and is tracked during the project. •***Subcontractor management***: The organization selects and effectively manages qualified software subcontractors. •***Quality assurance management***: All deliverables and process activities are reviewed and audited to verify that they comply with standards and guidelines adopted by the organization. •***Configuration management***: A set of configuration management items is defined and maintained throughout the entire project.

Mapping Process Maturity Level and Key Process Areas

Maturity Level	Key Process Area
3. Defined	Focus: Establish an infrastructure that allows a single effective software lifecycle model across all projects. •***Organization process focus***: The organization has a permanent team to constantly maintain and improve the software life cycle model. •***Organization process definition***: A standard software life cycle model is used for all projects in the organization. A database is used for software lifecycle related information and documentation. •***Training program***: The organization identifies the need for training for specific projects and develops training programmes. •***Integrated software management***: Each project team can tailor its specific process from the standard process. •***Software product engineering***: The software is built in accordance with the defined software life cycle, methods and tools. •***Intergroup coordination***: The project teams interact with each other to address requirements and issues. •***Peer reviews***: Developers examine each other's deliverables to identify potential defects and areas where changes are needed.

Mapping Process Maturity Level and Key Process Areas

Maturity Level	Key Process Area
4. Managed	Focus: Quantitative understanding of the software life cycle process and deliverables. •***Quantitative process management***: Productivity and quality metrics are defined and constantly measured across the project. It is critical that these data are not immediately used during the project, in particular to assess the performance of the developers, but are stored in a database to allow for comparison with other projects. •***Quality management***: The organization has defined a set of quality goals for software products. It monitors and adjusts the goals and products to deliver high quality products to the user.

Mapping Process Maturity Level and Key Process Areas

Maturity Level	Key Process Area
5. Optimized	Focus: Keep track of technology and process changes that may cause changes in the system model or deliverables, even during a project. •***Defect prevention***: Failures in past projects are analyzed, using data from the metrics database. If necessary, specific actions are taken to prevent those defects from occurring again. •***Technology change management***: Technology enablers and innovations are constantly investigated and shared throughout the organization. •***Process change management***: The software process is constantly refined and changed to deal with inefficiencies identified by the software process metrics. Constant change is the norm, not the exception.

Metrics

- Many software metrics have been proposed, such as the number of lines of source code, the number of branching points, the number of variables and operators, and the number of functional requirements.
- Metrics can also measure attributes of the process, such as deviations from the schedule, resources expended, and number of participants leaving prematurely.
  - Measuring financial status
  - Measuring technical progress
  - Measuring stability
  - Measuring modularity
  - Measuring maturity

Metrics

- Measuring Financial Status
  - Earned Value
- Measuring Technical Progress
  - Especially challenging in software development projects in determining exactly what tasks have actually been accomplished. Requires size metrics for each time.
    - For the development team this can include number of lines of source code under baseline, or the number of closed change requests.
    - For the testing team, this can include the number of defects found or the number of open change requests.
    - For the architecture team this can include the number of critical uses cases that have been demonstrated to work.
  - Size metrics, however, must be viewed in combination with quality metrics to ensure that participants are not locally optimizing their numbers.

Metrics

- Measuring stability
  - Once an initial version of the system is baselined, change requests are issued to control and track changes to the baseline.
  - Measuring the rate of new change requests corresponding to defects in the system indicates how stable the system becomes over time.
  - Measuring the rate of change requests to the requirements similarly indicates how stable the requirements have become in the view of the client.
  - If work completion indicates that the project is almost finished, but the rate of change requests is still high, the project might not be achieving its quality goals.

Metrics

- Measuring modularity
  - Breakage is a measure of the impact of a change on the baseline (which can be measured in terms of lines of code, number of files, function points, or any size metric that is appropriate in the context of the project).
  - It is expected that early changes affect more source code than later ones, as the architecture should become more modular over the project time.
  - Measuring breakage can encourage this trend and make visible early any architectural issues (e.g., identifying specific types of changes that yield high breakage).
  - Increasing breakage indicates a degrading modularity over the lifetime of the project, probably resulting from an inadequate architecture or specific nonfunctional requirements that are difficult to meet.

Metrics

- Measuring maturity
  - Mean time between failures measures the time between discovered defects during testing.
  - As the project nears delivery, the number of discovered defects should decrease, indicating an increasing maturity of the product.
  - A decreasing maturity can indicate that developers are under pressure to address open change requests and do not pay sufficient attention to quality.
  - Decreasing maturity can also result from lack of stability.

category	Examples	How Defined?
Project Performance	Budget Performance	Actual expenditures versus targets
	Schedule Performance	Rate of progress with milestone achievements
	Earned Value Performance	Value earned for milestones achieved versus budget set for work to be performed and actual expenditure
	Technical Performance	Indicators such as Requirements growth and size growth
Process Performance	Rework rate	Number of times it takes to get it right
	Defect rates	Number of defects discovered and removed as a function of time

category	Examples	How Defined?
Product Quality	Product Complexity	Cyclomatic number or some similar metric
	Defect Density	Number of defects as a function of size
Personnel Performance	Personal Productivity	Individual output as a function of the inputs used to generate them
Organizational Performance	Process maturity	SEI rating
	Product Quality	Compliant rate
	Productivity	Group output as a function of the inputs used to generate them
Enterprise performance	Profitability	Price to earnings ratio
	Return on equity	Earnings as a function of the capital used to generate it.
	Cost of sales	Money spent on sales as a function of revenue earned

Key Process Areas for Level 2: Repeatable

- Requirements Management
  - Goal 1: System requirements allocated to software are controlled to establish a baseline for software engineering and management use.
  - Goal 2: Software plans, products and activities are kept consistent with the system requirements allocated to software.
- Software Project Planning
  - Goal 1: Software estimates are documented for use in planning and tracking the software project.
  - Goal 2: Software project activities and commitments are planned and documented.
  - Goal 3: Affected groups and individuals agree to their commitments related to the software project.

Key Process Areas for Level 2: Repeatable

- Software Project Tracking and Oversight
  - Goal 1: Actual results and performance are tracked against the software plans.
  - Goal 2: Corrective actions are taken and managed to closure when actual results and performance deviate significantly from the software plans.
  - Goal 3: Changes to software commitments are agreed to by the affected groups and individuals.
- Software Subcontract Management
  - Goal 1: The prime contractor selects qualified software subcontractors.
  - Goal 2: The prime contractor and the software subcontractor agree to their commitments to each other.
  - Goal 3: The prime contractor and the software subcontractor maintain ongoing communications.
  - Goal 4: The prime contractor tracks the software subcontractor's actual results and performance against its commitments.

Key Process Areas for Level 2: Repeatable

- Software Quality Assurance
  - Goal 1: Software quality assurance activities are planned.
  - Goal 2: Adherence of software products and activities to the applicable standards, procedures and requirements is verified objectively.
  - Goal 3: Affected groups and individuals are informed or software quality assurance activities and results.
  - Goal 4: Noncompliance issues that cannot be resolved within the software project are addressed by senior management.
- Software Configuration Management
  - Goal 1: Software configuration management activities are planned.
  - Goal 2: Selected software work products are indentified, controlled and available.
  - Goal 3: Changed to identified software work products are controlled.
  - Goal 4: Affected groups and individuals are informed of the status and content of software baselines.

Key Process Areas for Level 3: Defined

- Organization Process Focus
  - Goal 1: Software process development and improvement activities are coordinated across the organization.
  - Goal 2: The strengths and weaknesses of the software processes used are identified relative to a process standard.
  - Goal 3: Organization-level process development and improvement activities are planned.
- Organization Process Definition
  - Goal 1: A standard software process model for the organization is developed and maintained.
  - Goal 2: Information related to the use of the organization's standard software process model by its software projects is collected, reviewed and made available.

Key Process Areas for Level 3: Defined

- Training Program
  - Goal 1: Training activities are planned.
  - Goal 2: Training for developing skills and knowledge to perform software management and technical roles is provided.
  - Goal 3: Individuals in the software engineering group and software-related groups receive the training necessary to perform their roles.
- Integrated Software Management
  - Goal 1: The project's defined software process model is a tailored version of the organization's standard software process model
  - Goal 2: The project is planned and managed according to the project's defined software process.

Key Process Areas for Level 3: Defined

- Software Product Engineering
  - Goal 1: The software engineering tasks are defined, integrated, and consistently performed to produce the software.
  - Goal 2: Software work products are kept consistent with each other.
- Intergroup Coordination
  - Goal 1: The customer's requirements are agreed to by all affected groups.
  - Goal 2: The commitments between the engineering groups are agreed to by the affected groups.
  - Goal 3: The engineering groups identify, track and resolve intergroup issues.
- Peer Reviews
  - Goal 1: Peer review activities are planned.
  - Goal 2: Defects in the software work products are identified and removed.

Key Process Areas for Level 4: Managed

- Quantitative Process Management
  - Goal 1: The quantitative process management activities are planned.
  - Goal 2: The process performance of the project's defined software process is controlled quantitatively.
  - Goal 3: The process capability of the organization's standard software process is known in quantitative terms.
- Software Quality Management
  - Goal 1: The project's software quality management activities are planned.
  - Goal 2: Measurable goals for software product quality and their priorities are defined.
  - Goal 3: Actual progress toward achieving the quality goals for the software products is quantified and managed

Key Process Areas for Level 5: Optimizing

- Defect Prevention
  - Goal 1: Defect prevention activities are planned.
  - Goal 2: Common causes of defects are sought out and identified.
  - Goal 3: Common causes of defects are prioritized and systematically eliminated.
- Technology Change Management
  - Goal 1: Incorporation of technology changes are planned.
  - Goal 2: New technologies are evaluated to determine their effect on quality and productivity.
  - Goal 3: Appropriate new technologies are transferred into normal practice across the organization
- Process Change Management
  - Goal 1: Continuous process improvement is planned.
  - Goal 2: Participation in the organization's software process model improvement activities is organization wide.
  - Goal 3: The organization's standard software process model and the project's defined software processes are improved continuously.

Project Audits

What is an audit?

- Independent review and examination of records and activities to assess the adequacy of system controls, to ensure compliance with established policies and operational procedures, and to recommend necessary changes in controls, policies, or procedures.
- Can be applied to almost anything that is auditable: financial audit, quality audit, project audit, etc.
- It is expected that the opinion of the auditor as expressed in the final audit report will be objective.

Auditing a project

- The main aim of an audit is to reduce uncertainty.
- An audit is an official examination. In project work, we are concerned with two types of audit:
  - A general project audit to determine the status of the project
  - A quality audit to determine how closely the processes adhere to standards and how closely the products match the user's requirements.

Who does the auditing?

- There are two types of auditors:
  - Internal auditor – These are employees of a company who assess and evaluate its system of internal controls. To maintain independence they present their reports directory to the Board of directors or top management.
  - External auditor – These are independent staff assigned to assess and evaluate an organization's systems of internal control or to perform other agreed upon evaluations. They are called upon from outside the company.

Who does the auditing?

- Auditors must be independent from and external to the group being audited so that there can be no accusations of bias or temptation to justify the status quo. Their function is to discover the truth.
- Usually a small team is required, typically three or four skilled people.
- Although auditors must be external to the group being audited they need not be external to the organization. Audits can be either internal or external.

Project audit

- A project audit, in contrast to a financial audit, is an appraisal of the technical status of a project, or a specified part of a project, and will most usually be performed during a project rather than at its end.
- A project audit is performed by an independent auditing body external to the project team, and is conducted in a formal and systematic manner.

Project audit

- Purpose of a project audit
  - Determine if the project is on-track or not.
  - If the project is off-track, determine why this is the case and offer specific advice on how to get it back on track or advice on whether the project ought to be cancelled.
  - Determine whether the causes of the project issues are systemic and ascertain how these can be avoided (obviated) in the future.

Aspects to cover

- Project Management deliverables
- All project deliverables
- Subcontractor contracts
- Etc…

Quality audit

- Usually carried out for the specific purpose of checking that quality standards are being applied when:
  - A quality problem is suspected, and a manager would appreciate a fresh look at the area by an impartial observer
  - A company's business requirements are such that it must be registered as conforming to an external standard. This may be to conform with the client's requirements, such as conditions for tendering for government contracts. In such cases internal audits are necessary to prepare the company for audits by the registering body.

When to audit

- Usually an audit is a short, sharp exercise which might in some cases take as little as a day, but typically takes 2-3 weeks.
- Audits may arise due to:
  - Having been planned at the outset of the project
  - The project appears to be in difficulty
- Projects are usually in progress when an audit takes place. Audits are more useful as a tool for assessing the progress of a current project than for reporting on the achievements of a completed one.
- Unlike a financial audit, project auditors are often expected to make recommendations, or at least clarify the status. The future conduct of the project may be materially affected by the auditor's actions.
- An audit is expected to reveal not only current status, but also likely future prospects so that the recommendations of the report can influence the future conduct of the project. An essential feature of the audit is the comparison with project requirements; the auditors have to consider not only the current status but also the project's expectations as expressed in the contract.

How is auditing done

- An audit is formal and systematic. It will be initiated formally and conducted with a prescribed scope, to produce a report within a given time.
- Systematic methods ensure that the conclusions of the report are properly justified as confirmed facts, free from subjective judgments.
  - The audit must be focused on a particular topic (a process, procedure or deliverable)
  - The criteria against which the audit is made must be identified
  - The audit team must be properly briefed and must be given access to all relevant documents and personnel
  - The audit team must be given adequate time and resources to produce a proper report, but reports should be prepared within an agreed timeframe.
  - There should be no prior indication from management as to what results they expect.
  - The report from the audit team should be reviewed with the relevant managers (project manager, general manager, quality manager)
  - Any disagreements should be recorded in the report

How is auditing done

- An audit may be considered to consist of the following six phases:
  - Phase 1
    - Initiation and planning – agree the purpose, constraints, terms of reference
  - Phase 2
    - Establish project baseline – what was expected at this point according to the contract and all agreed changes
  - Phase 3
    - Investigation – gather the facts and record where they have come from
  - Phase 4
    - Analysis – try to draw conclusions from the data
  - Phase 5
    - Report and present conclusions – report to the requester of the audit and other parties as agreed at the outset
  - Phase 6
    - Termination – evaluate the audit

Initiation and Planning

- An agreement is arrived at with the project sponsor covering:
  - The scope of the audit (what's in, what's not?
  - The Basis of the Opinion
  - Documents and References
  - The questions to be asked

Enquiry

- Phase 2
  - Obtain the project organization and structure
  - Establish project baseline – what was expected at this point according to the contract and all agreed changes
- Phase 3
  - Investigation – gather the facts and record where they have come from
  - Identify who should be interviewed
  - Obtain all identified documents and references
  - Conduct interviews
    - Others may be identified during the interviews
  - Review all documents and references
    - May be added to as the audit progresses
- Phase 4
  - Analysis – try to draw conclusions from the data

Reporting

- Daily/Weekly updates on progress/issues for project sponsor
- Identify obstacles and how to overcome them.
- Provide the sponsor with critical project findings as soon as they can be substantiated.
- Provide a draft audit report to the sponsor before the end of the audit to gauge their acceptance on key findings and wording.

Structure of an audit report

- Executive Summary
  - Audit Focus
  - Basis of Opinion
  - Audit Findings
  - Recovery Plan
- Documents and References Used (Overall)
- Individual Audit areas with individual findings
  - Area of Interest and Basis of Opinion
  - Documents and References used
  - Statements of Fact
  - Findings

Audit Focus

- A clear statement on what is being audited :
  - For example: “The focus of the audit was on the project management deliverables:
    - Project plan
    - Project schedule
    - Work breakdown structure
    - Etc.
  - As well as the core project deliverables:
    - Systems Integration Architecture and Strategy
    - Business Process Model
    - Developed Software
    - Etc.”

Basis of Opinion

- For instance for a financial audit
- “We conducted our audit in accordance with International Standards on Auditing. Those standards require that we plan and perform the audit to obtain reasonable assurance on whether the financial statements are free of material misstatement. An audit includes examining, on a test basis, evidence supporting the amounts and disclosures in the financial statement. An audit also includes assessing the accounting principles used and significant estimates made by management, as well as evaluating the overall financial statement presentation. We believe that our audit provides a reasonable basis for our opinion”

Basis of Opinion for a project audit

- “The audit was conducted based on normal industry expectations for the area being audited as follows:
  - The core project management deliverables were audited for conformance to the Project Management Institute’s Project Management Body of Knowledge (PMBOK) at a basic level only for the Project Plan, the WBS, the Project Schedule, Project Estimates, Resourcing, and Project Change Control”
  - The core project deliverables were assessed according to whether they in fact were identified as being required and if they were, what would they be reasonably be expected to cover. The documents and references that were used are fully listed at the beginning of the audit areas, as well as where they are individually referenced. They were found at <> as well as on the server that hosted the referenced application.
  - Etc.
- Every attempt was made to conduct the audit itself and to present the audit findings without prejudice.”

Audit Findings

- Clear statements that establish the true state of the project and its core deliverables as compared to what was previously reported.
- Must stick to facts that can be proved – i.e. a professional (objective) rather than a personal (subjective) opinion.
- Must be such that any other person of the same qualifications for the area of findings would have reasonably concluded the same thing.

Recovery Plan

- If the project is to be continued, it is likely that a recovery plan will be proposed. It should
  - Highlight what can be carried forward.
  - Be honest about what cannot be carried forward.

Human Resource Management

Objectives

- Human Resource Management
- Why do people work?
- What power does a Project Manager have?
- How are people motivated?
- How can teams be managed?

Human Resource Management

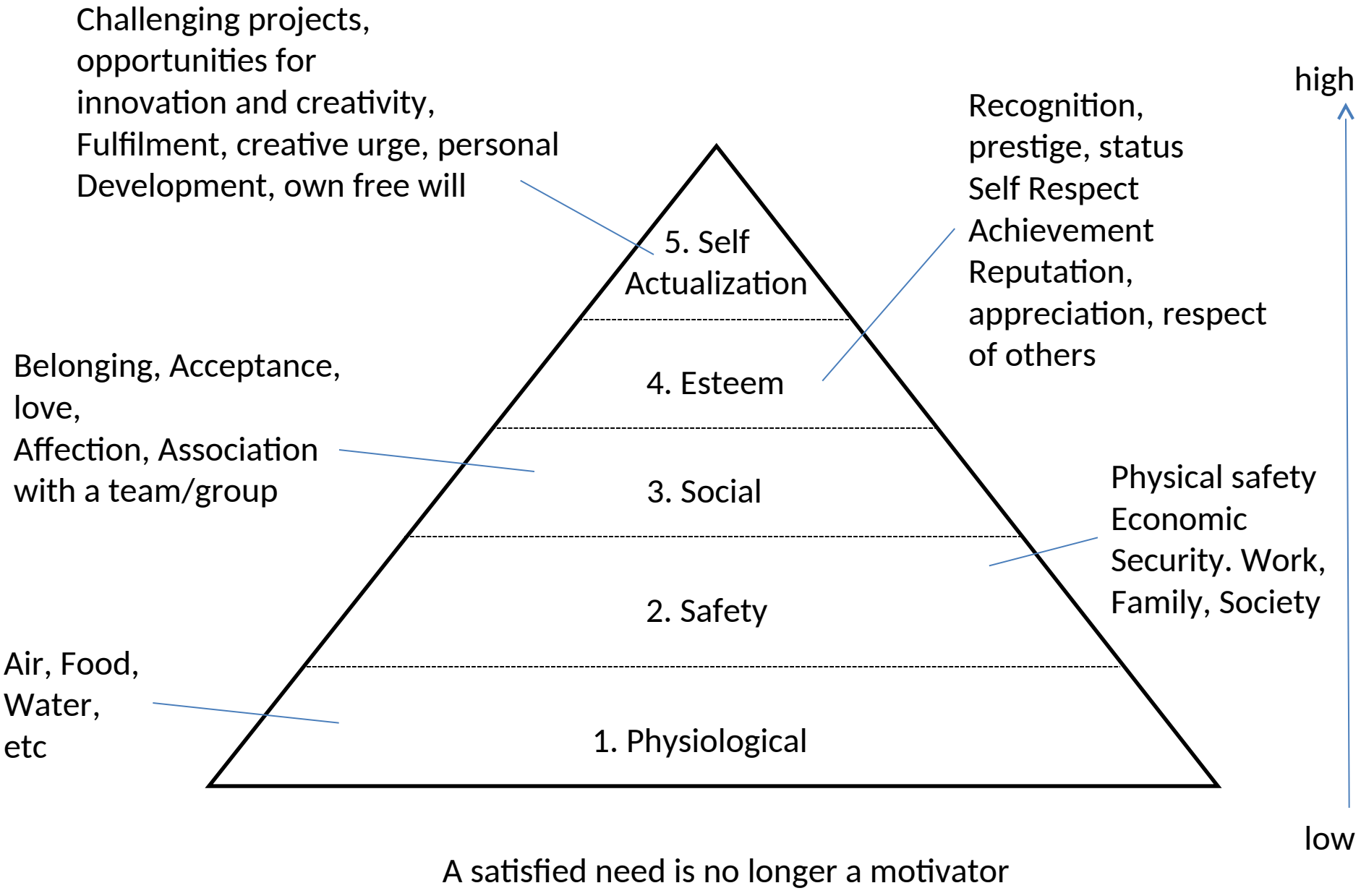
- Making effective use of the people involved with a project.
- Four Processes:
  - Human Resources Planning
    - Identify and list project roles and responsibilities
  - Acquiring the project team
    - Getting the right people assigned to the project team.
  - Developing the project team
    - Building individual and group skills to enhance project performance.
  - Managing the project team
    - Tracking team member performance, motivating team members, providing timely feedback, resolving issues and conflicts, and coordinating changes to enhance project performance

Why people work

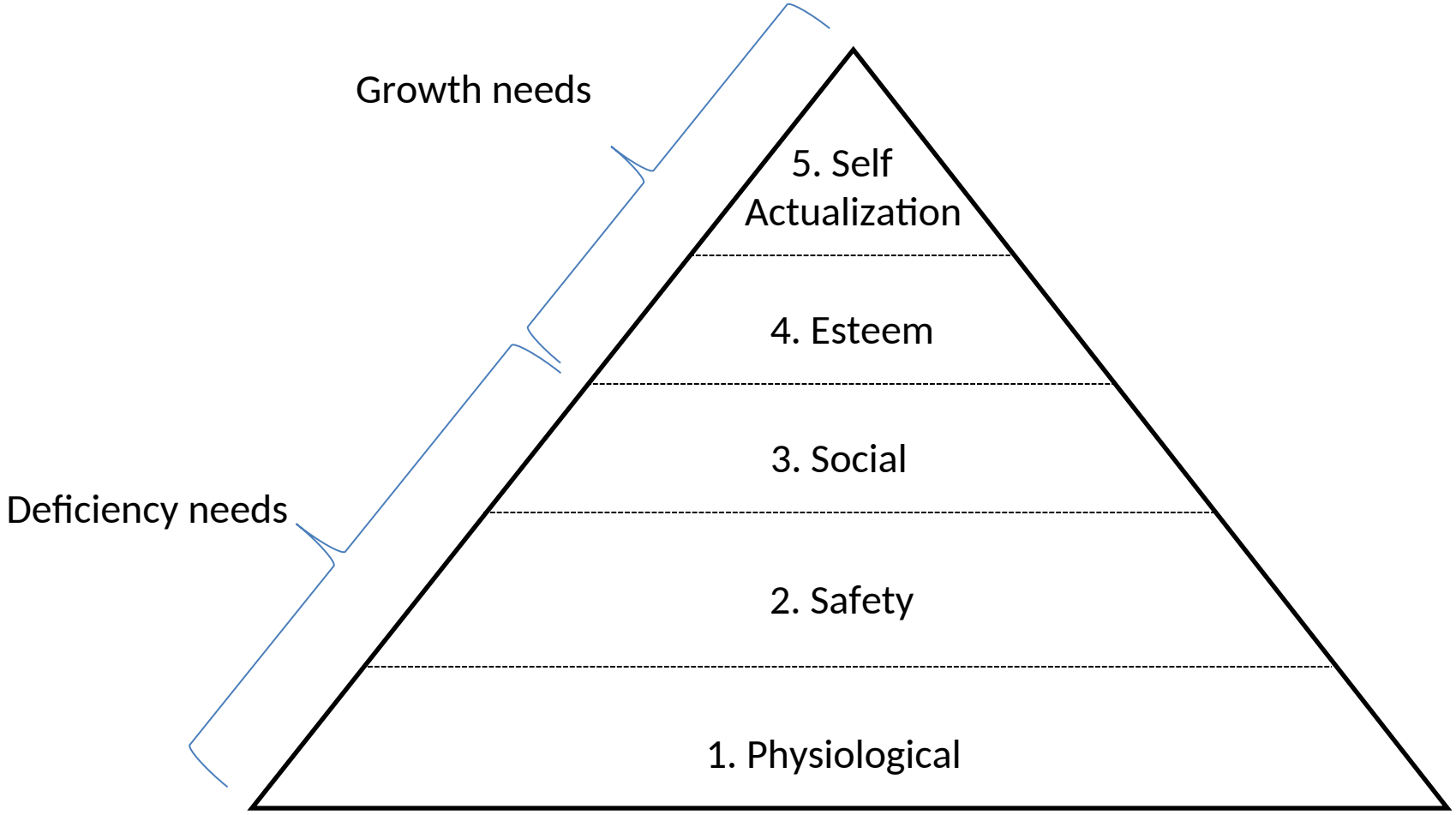
- To manage people, we need to understand why they work for us in the first place.
- Why do people work?
- What do they need so they can work better?
- Much research by psychologists and management theorists have been devoted to the field of managing people at work. Important areas related to project management include:
  - Motivation theories
  - Influence and power
  - Effectiveness

Maslow's Hierarchy of Needs

- Abraham Maslow (A Theory of Human Motivation) suggested that human beings possess unique qualities that enable them to make independent choices, thus giving them control over their destiny.
- He developed a hierarchy of needs, which states that people's behaviors are guided or motivated by a sequence of needs.
- These needs are arranged in layers. Bottom layer needs must be met before people are concerned with the next layer above.



A satisfied need is no longer a motivator



A satisfied need is no longer a motivator

- Provides a structure allowing managers to understand whether an employees needs are being met. All or not all.
- Do managers need to satisfy of an employee's needs? Perhaps only deficiency needs need to be met at work, with growth needs met elsewhere via activities outside of work. This depends on various factors such as employee ambition, nature of job, country of work and manager's approach.

Criticism

- Does not appear to work in all cases
- Cultural differences may lead to misapplication
- Scant real-world evidence to support its application. (i.e. not scientific?).

Herzberg's Motivational and Hygiene factors

- Frederick Herzberg wrote in the area of worker motivation. He had a Two-Factor Theory. He distinguished between:
  - Motivational factors
    - Achievement, recognition, the work itself, responsibility, advancement and growth. These factors produce job satisfaction.
  - Hygiene factors (Maintenance Factors)
    - Larger salaries, more supervision and an attractive work environment. These factors cause dissatisfaction if not present, but do not motivate workers to do more.

Herzberg's Motivational and Hygiene factors

Hygiene factors	Motivational factors
Security, Status, Relationship, Personal Life	Sense of Achievement
Larger salaries	Recognition (by managers and colleagues)
More supervision	Work itself (meaning and interesting work)
More attractive work environment	Responsibility
Computer or other required equipment	Advancement
Health benefits	Growth opportunities
Training	
Company policies and administration	

- Hygiene factors must be covered before moving to the motivational factors.
- Hygiene factors: reduce negative attitudes and dissatisfaction. Extrinsic components.
- Motivational factors: create positive attitude and satisfaction. When used in the right way, employees can feel that they have made a difference. Intrinsic components.

Two-Factor Theory

- Fix Company policies
- Provide suitable supervision
- Create a culture of respect
- Competitive wages
- Build job status
- Provide job security
- Provide opportunities for achievement and advancement
- Create work
- Give responsibility
- Allow people to pursue their dream jobs

Criticism

- Overlooks variables among individuals.
- Assumes a correlation between satisfaction and productivity.
- No measure of satisfaction was settled on.
- During development this only looked at white collar workers. The factors applicable to them do not necessarily apply to other workers.

Intrinsic and Extrinsic Motivation

- Intrinsic motivation
  - Causes people to participate in an activity for their own enjoyment.
- Extrinsic motivation
  - Causes people to do something for a reward or to avoid a penalty

Intrinsic vs. Extrinsic

- People are usually attracted to a job for extrinsic reasons (good pay, good benefits, good working conditions)
- People usually stay with a job for intrinsic reasons (they like the people they work with, they find the job challenging, people praise them for the work they do)
- Paying people more money (extrinsic motivation) does not make them work harder. But it might keep them from leaving and it might attract new employees)
- Intrinsic motivation is what builds loyalty and dedication in your employees

McGregor's Theory X and Theory Y

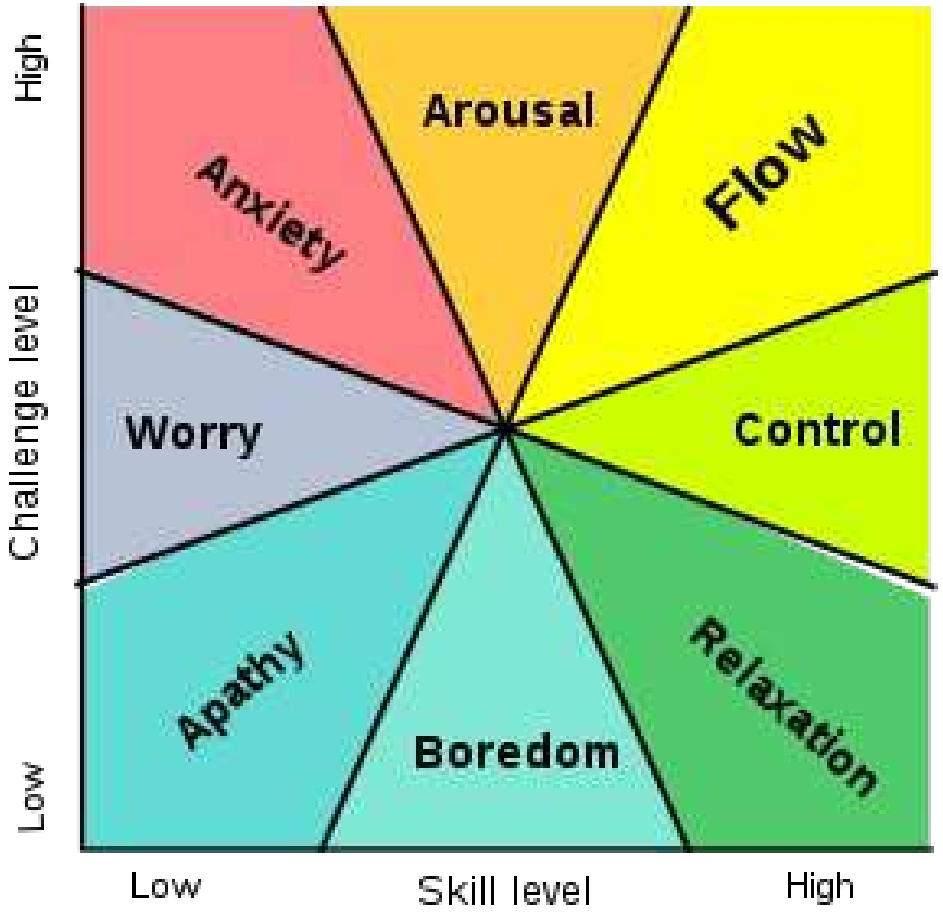
- Douglas McGregor popularized the human relations approach to management (1960s).
  - Theory X
    - Assumes workers dislike and avoid work, so managers must use coercion, threats of punishment and sanction, and various control schemes to get workers to meet objectives.
  - Theory Y
    - Assumes individuals consider work as natural as play or rest and enjoy the satisfaction or esteem and self-actualization needs.
- Theory Z
  - Introduced in 1981 by William Ouchi, based on the Japanese approach to motivating workers, emphasizing trust, quality, collective decision making, and cultural values


X
Y

- Theory X and Theory Y are two sets of assumptions that managers make about their employees.
- McGregor (a humanistic psychologist) felt Theory Y was more modern and applicable. Theory X was considered by him to be old fashioned and associated with the earlier part of the 20th century.
- It is seen as a spectrum along which individual managers can choose to operate.

Flow Theory

- Mihaly Csikszentmihalyi



Flow State

- Intense and focused concentration on the present moment
- Merging of action and awareness
- A loss of reflective self-consciousness
- A sense of personal control or agency over the situation or activity
- A distortion of temporal experience, as one's subjective experience of time is altered
- Experience of the activity as intrinsically rewarding, also referred to as autotelic experience

Achieving Flow

- The flow state is an optimal state of intrinsic motivation, where a person is fully immersed in their activity.
  1. There are clear goals every step of the way.
  2. There is immediate feedback to one's actions.
  3. There is a balance between challenges and skills.
  4. Action and awareness are merged.
  5. Distractions are excluded from consciousness.
  6. There is no worry of failure.
  7. Self-consciousness disappears.
  8. The sense of time becomes distorted.
  9. The activity becomes an end in itself.

The power of a project manager

- A project manager must have power to do their job.
- Managers must understand the types of power they have
- Some power is better to use than others
- Power that builds trust, respect and affirms people is best

Types of Power

- Power is the potential to influence behavior to get people to do things they otherwise would not do.
- Types of power include:
  - Legitimate
    - The ability to claim legitimate power as a reason to earn support. “Do it because I am your boss”
  - Coercive/Penalty
    - The ability to gain support through the use of force, threats or fear
  - Reward
    - Ability to gain support because subordinates perceive someone as being capable or directly or indirectly dispensing valued organizational rewards (e.g. salary adjustments, promotion, bonus, future work assignments, etc)
  - Expert
    - The ability to gain support because of superior knowledge, skills, or understanding of the problem or situation.
  - Referent
    - Ability to gain support because people feel personally attracted to the manager

Thamhain & Wilemon’s ways to have Influence and Power on projects

1. **Authority**: The legitimate hierarchical right to issue orders.
2. **Assignment**: The project manager's perceived ability to influence a worker's later work assignments.
3. **Budget**: The project manager's perceived ability to authorize others' use of discretionary funds.
4. **Promotion**: The ability to improve a worker's position.
5. **Money**: The ability to increase a worker's pay and benefits.
6. **Penalty**: The project manager's ability to cause punishment.
7. **Work challenge**: The ability to assign work that capitalizes on a worker's enjoyment of doing a particular task.
8. **Expertise**: The project manager's perceived special knowledge that others deem important.
9. **Friendship**: The ability to establish friendly personal relationships between the project manager and others.

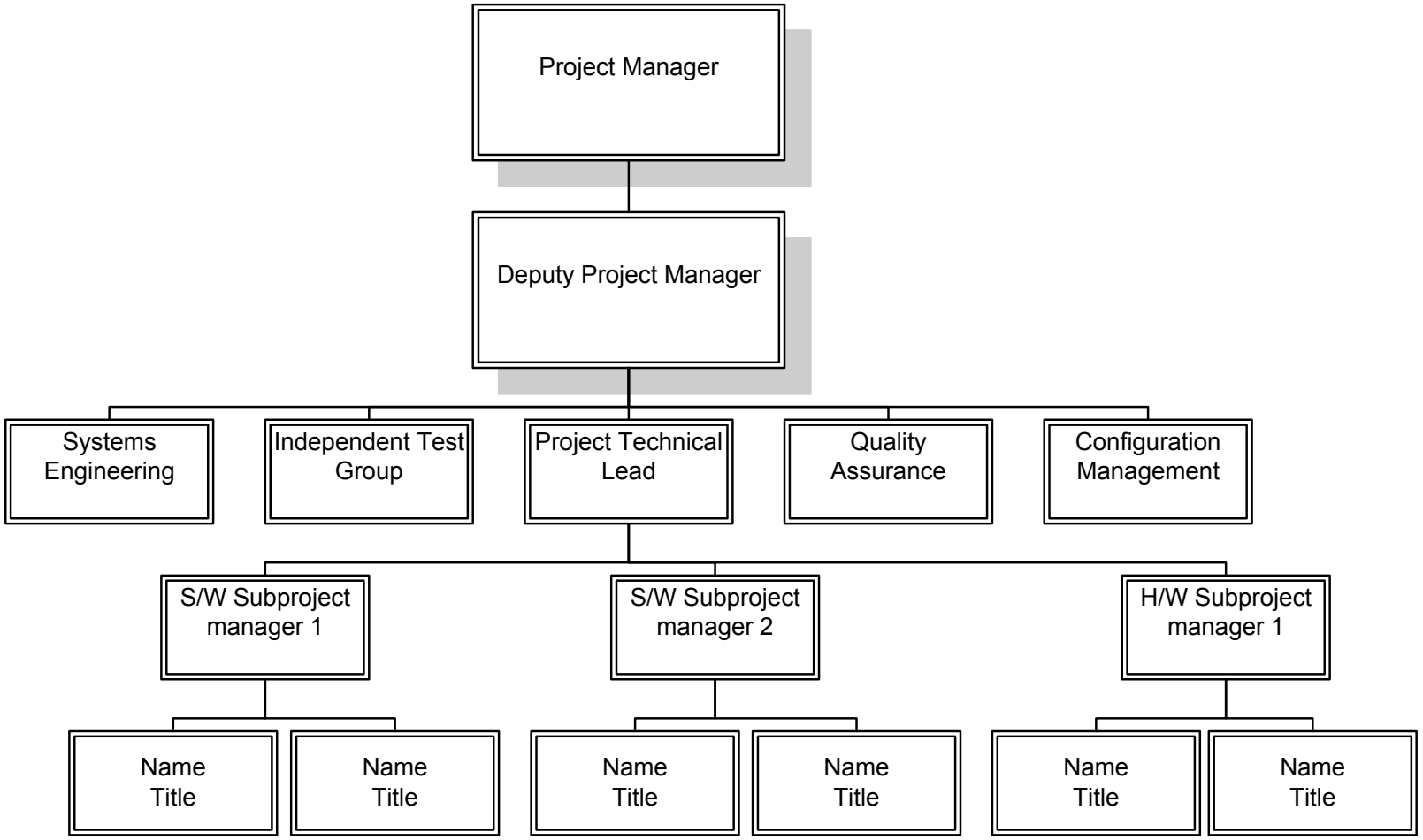
Using Power for Success

- Projects are more likely to succeed when project managers influence people using:
  - Expertise
  - Work challenge
- Projects are more likely to fail when project managers rely too heavily on:
  - Authority
  - Money
  - Penalty

Human Resource Planning

- Involves identifying and documenting project roles, responsibilities, and reporting relationships
- Outputs include:
  - Project organization charts
  - Staffing management plan
  - Responsibility assignment matrices
  - Resource histograms

Sample Organization Chart



Responsibility Assignment Matrices

- A responsibility assignment matrix (RAM) is a matrix that maps the work of the project as described in the work breakdown structure to the people responsible for performing the work.
- It can be created in various ways to meet unique project needs.

Sample RAM

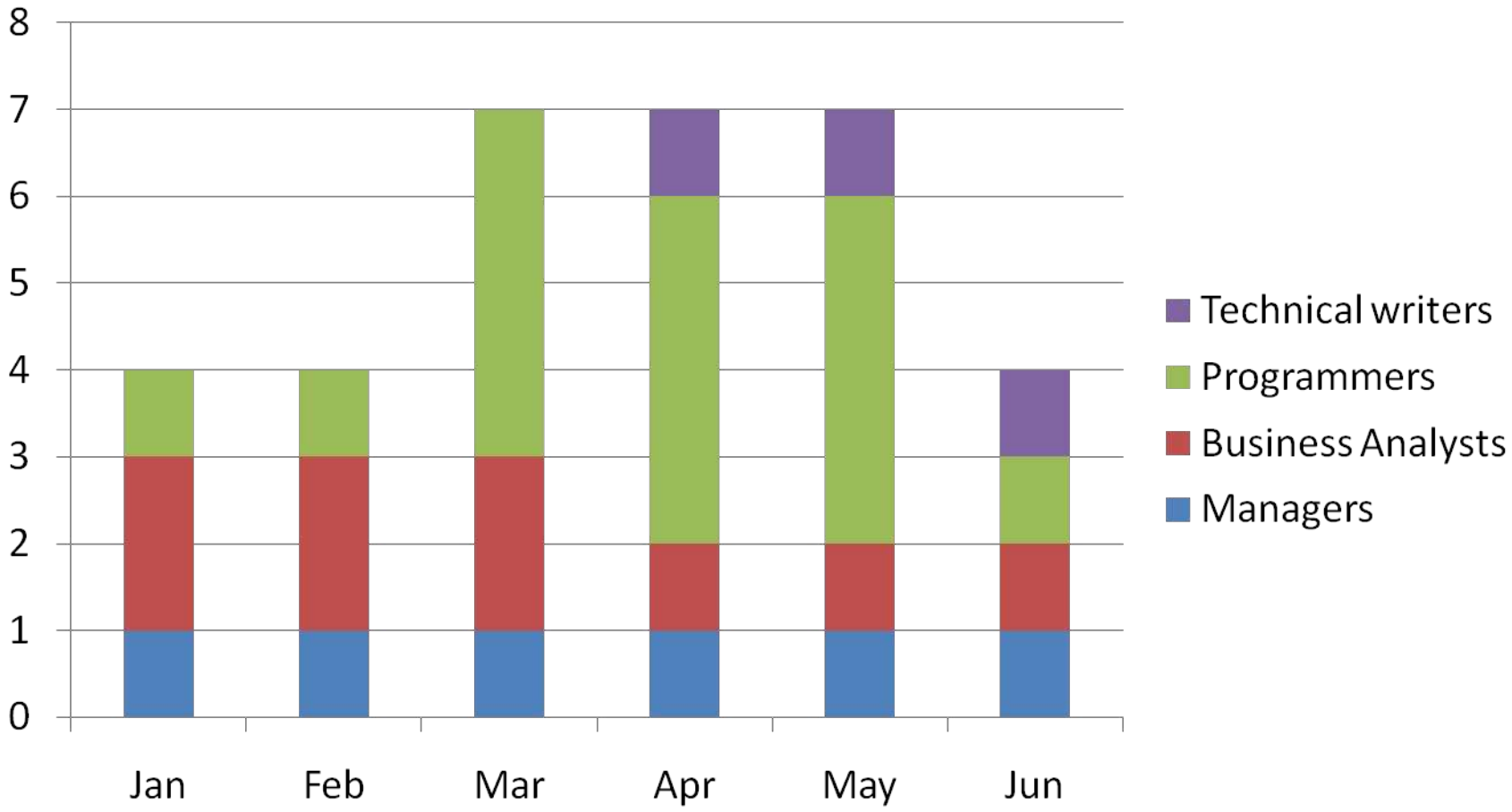
Tasks	Subtasks	SE	HA	SA	TR	User
A	System Design					
A1	Define work process	X				X
A2	Identify required information and sources	X				X
A3	Design software system to support work process					
A31	Basic software	X				
A32	Data access required	X				X
A33	Vendor software required	X				X
A4	Identify required hardware components	X				X
A0	System design milestone	X				X

Key:
- SE System engineer personnel
- HA Hardware acquisition
- SA Software acquisition
- TR Trainer

Staffing Management Plans

- A staffing management plan describes when and how people will be added to and taken off the project team.
- A resource histogram is a stacked column chart that shows the number of resources assigned to a project over time.

Sample Resource Histogram



Acquiring the project team

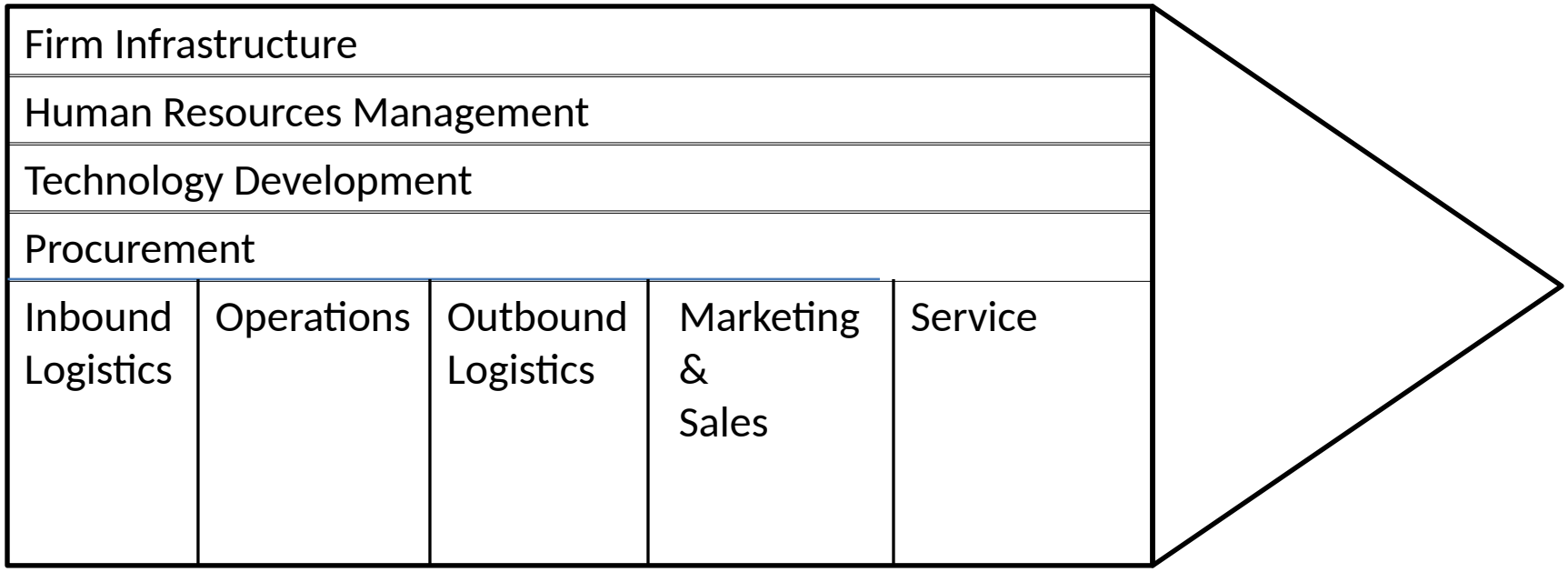
- Acquiring qualified people for teams is crucial.
- Staffing plans and good hiring procedures are important, as are incentives for recruiting and retention.
- Why people leave their jobs
  - They feel that they do not make a difference
  - They do not get proper recognition
  - They are not learning anything new or growing as a person
  - They do not like their coworkers
  - They want to earn more money

Training

- Training can help people understand themselves and each other, and understand how to work better in teams.
- Team building activities include:
  - Physical challenges
  - Psychological preference indicator tools

Project Procurement

Porter’s Value Chain



Porter’s value chain showing support activities and primary activities.

Key factors in evaluating Vendors for MIS

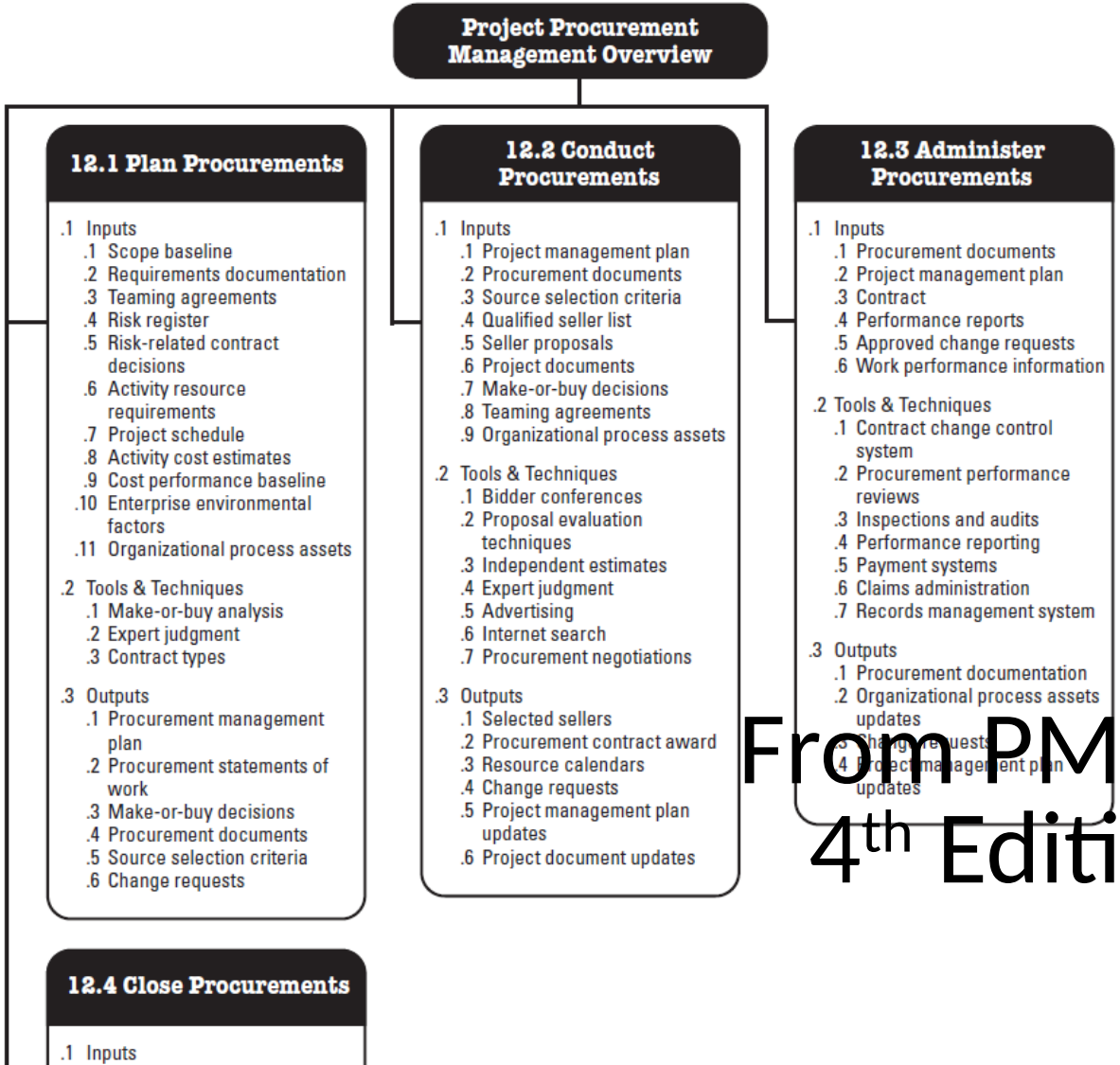
- Possess long-range plans for support of the product.
- Possess a good corporate reputation for reliability
- Reflect financial stability
- Possess a high credibility rating in the industry
- Provide staff with in-depth industrial knowledge and background
- Possess an intimate knowledge of their product
- Be totally committed to the support of the product
- Be easily accessible and available to clients
- Provide an effective implementation program
- Provide additional services as needed to support the product
- Maintain effective communication with users
- Encourage user interaction and involvement
- Provide for a well-defined maintenance plan

Project Procurement Management

- The processes related to the purchase of products, services or results necessary to perform the work.
- Includes contract management and change control processes required to administer contracts or purchase orders issued by authorized project team members.
- Also includes administering any contract issued by an outside organization (the buyer) that is acquiring the project from the performing organization (the seller), and administering contractual obligations placed on the project team by the contract

Project Procurement Management

- Plan purchases and acquisitions
  - Determining what to purchase and determining when and how to do so.
- Plan contracting
  - Documenting products, services and results requirements and identifying potential sellers
- Request seller responses
  - Obtaining information, quotations, bids, offers or proposals
- Select sellers
  - Reviewing offers, choosing among potential sellers and negotiating a written contract with each seller
- Contract administration
  - Managing the contract and relationship between the buyer and seller, reviewing and documenting how a seller is performing or has performed to establish required corrective actions and provide a basis for future relationships with the seller. Managing contract related changes and managing the relationship with the outside buyer of the project
- Contract closure
  - Completing and settling each contract including the resolution of any open items, and closing each contract applicable to the project or a project phase.



From PMBOK 4th Edition

Contract

- A mutually binding agreement that obligates the seller to provide the specified products, services or results and obligates the buyer to provide monetary or other valuable consideration.
- A contract is a legal relationship subject to remedy in the courts. Some agreements are simple others are complex. Contracts can reflect the simplicity or complexity of the deliverables.
- Depending on the application area, contracts can also be called an agreement, subcontract, or purchase order.
- Most organizations have well documented policies and procedures specifying who can enter into agreements on the organization's behalf.

Plan purchases and acquisitions

- The Plan Purchases and Acquisition process provides a pattern which can be used to determine which needs are better met by using an outside organization versus the project team. Common inputs to this decision is when the result is needed and how best to obtain the result.
- As with all processes, there are inputs to the Plan Purchases and Acquisitions Process. These inputs provide the foundation for determining what, when, and how to purchase. These input are given next.

Inputs

Enterprise Environmental Factors – Enterprise environmental factors concerning mark conditions. What products, services, and results are available in the marketplace? Where can you obtain them? Are there terms and conditions which are applicable? Is there a legal or purchasing department? Or is that the project teams responsibility?

Organizational Process Assets – Organizational process assets depicts whether formal or informal policies and procedures exist for dealing with procurement within the organization. Some common types of organizational process assets are, all requisitions for purchase orders must be signed by the CIO. All purchase requisitions above $2000 must be initialed by all EVPs. All vendors must have been in business for a period of at least 3 years.

Project Scope Statement – Project Scope Statement details the projects boundaries, requirements, constraints, and assumptions. Check out the constraints in the project scope statement to see if there are any required delivery dates or need for skilled resources.

Work Breakdown Structure – The Work Breakdown structure illustrates how each work package's interdependencies.

WBS Dictionary – The WBS Dictionary details the work needed to complete each WBS package. Check out the contract info and quality needs when reviewing for the planning and acquisition process.

Project Management Plan – Check out the following items: Risk register, Risk related contractual agreements, Activity resource requirements, Project schedule, Activity cost estimates, Cost baseline.

- **Make-or-buy Decisions** - Make-or-buy analysis involves comparing the relative benefits of in-house versus out-sourcing production of a given product, service, or result. Common considerations are if the skill set in house is capable of delivering the product or service and if the same quality or a higher standard can be obtained from outside.

- **Expert Judgment** - Expert judgment brings in specialized advice, such as technical, legal, or market knowledge, to inform buying decisions. Sometimes using experts can bring about new approaches which simply were not considered without the expertise or experience of being one that has already been there done that.

- **Contract Types** - Contract Types are the types of contracts available for use. Different contracts are used for different purposes and intents. Generally there are three main categories:

Contract Types

Contract Types - Different contracts are used for different purposes and intents. Generally there are three main categories:

- **Fixed-price or lump-sum contracts** – Generally involves a total price for a well-defined product. Fixed-price contracts may include incentives for early delivery. Buying shrink wrapped software is commonly a fixed price contract.

- **Cost-reimbursable contracts** – Generally involves payment to the vendor for actual cost plus a fee for profit. Actual costs comprise direct and indirect cost of making the product or providing the service. There are three common types of cost-reimbursable contracts.
  - **Cost-Plus-Fee (CPF) or Cost-Plus-Percentage of Cost (CPPC)** – the seller is reimbursed for the allowable costs and receives a percentage of the costs as a fee.
  - Cost-Plus-Fixed-Fee (CPFF) – the vendor is reimbursed for allowable costs and receives a fixed fee. The fee normally doesn't change unless scope changes allocate a new fee.
  - **Cost-Plus-Incentive-Fee (CPIF)** – Cost-Plus-Incentive reimburses the seller for allowable costs and allocates a predetermined fee, as an incentive for achieving certain performance objectives.

- **Time and Material contracts** –a hybrid combination of cost-reimbursable and fixed price contracts. A common indicator is their open-ended nature. The full value of the agreement and the exact quantity of items to be delivered are not defined at the time of

Outputs

Procurement Management Plan – The procurement management plan explains how procurement will be managed from inception to closing. Commonly the procurement management plan details the types of contracts to be used, who is accountable for estimates, the guidelines for the build or buy decision. What are the common lead times and how best to handle them? Finally what is the form and format of the statement of work?

Contract Statement of Work (SOW) - The contract statement of work explains the items being purchased with enough detail and clarifications for everyone to be in agreement with what is being purchased. The contract statement of work should have a clear concise description of the services needed and operational support.

Make-or-buy Decisions – Make or buy decisions are pretty self explanatory as to the purpose. However, it is good to have a document with a list of the justification for the decision and responsible party.

Requested Changes – Requested changes to the plan should be placed into the integrated change control process.

Plan Contracting

- Prepares the documents need to support the Request Seller Responses process and Select sellers process

Inputs

- Procurement management plan
- Contract statement of work
- Make or buy decisions – list of items to be purchased those to be produced by the project team
- Project management plan
  - Risk register – risk related information such as identified risks, root causes of risks, risk owners, risk analysis results, risk prioritization, risk categorization and risk responses generated by the risk management process.
  - Risk related contractual agreements – agreements for insurance, services etc that are prepared to specify each party's responsibility for specific risks.
  - Resource requirements
  - Project schedule
  - Activity cost estimate
  - Cost baseline

Tools and techniques

- Standard forms – include standard contracts, standard descriptions of procurement items, non-disclosure agreements, proposal evaluation criteria checklists, or standardized versions of all parts of the needed bid documents.
- Expert judgement - often required to assess the inputs to and outputs from a process. Expert purchasing judgement can also be used to evaluate offers or proposals made by sellers. Expert legal judgement may involve the services of a lawyer to assist with non-standard procurement terms and conditions.

Outputs

- Procurement documents – used to seek proposals from prospective sellers. A term such as bid, tender, or quotation is generally used when the seller selection decision will be based on price (when buying commercial or standard items), while a term such as proposal is generally used when other considerations, such as technical skills or technical approach, are paramount. Common names are “invitation to bid,” “request for proposal,” “request for quotation,” “tender notice,” “invitation for negotiation,” and “contractor initial response.” Procurement documents are structured to facilitate an accurate and complete response from prospective sellers and to facilitate easy evaluation of the bids.
- Evaluation criteria – used to rate or score proposals. Can be objective or subjective. Can be limited to purchase price, but often other criteria need to be taken into consideration. E.g. understanding of need, overall or life-cycle cost, technical capability, management approach, technical approach, financial capacity and interest, references, intellectual property rights, proprietary rights.
- Contract statement of work (updates)

Request Seller Responses

- Obtains responses such as bids and proposals, from prospective sellers on how project requirements can be met. The prospective sellers, normally at no direct cost to the project or buyer, expend most of the actual effort in this process.

Inputs	Tools and Techniques	Outputs
1. Organizational process assets 2. Procurement management plan 3. Procurement documents	1. Bidder conferences 2. Advertising 3. Develop qualified sellers list	1. Qualified sellers list 2. Procurement document package 3. Proposals

Inputs

- Organization Process Assets – some organizations maintain lists or files with information on prospective and previously qualified sellers (sometimes called bidders) who can be asked to bid, propose, or quote on work. These lists will generally have information on relevant past experience and other characteristics of the prospective sellers.
- Procurement management plan – describes how the procurement process will be managed from developing procurement documentation through to contract closure. Possible includes: contract types, who prepares estimates, standardized procurement documents, managing multiple providers, coordinating procurement with other project aspects like scheduling, etc.

Software Estimation

Estimation

1. COCOMO approach which involves making an educated guess at how many thousands of source code lines one would need to complete a software development exercise.
2. Function Points approach involves a count of the number of functional items in a software development exercise. This count is then used to make an assessment of the complexity of the development task and serves as a basis for estimates of effort, time and money needed to complete the exercise.

COCOMO '81

Constructive Costing Model

Barry Boehm

Historical model ——> Cocomo II
Cocomo '81 Modern version of Cocomo 81

- Suitable for batch systems (CL) common with COBOL etc.

- suitable for modern software architecture

Basic

Using the Cocomo 81 model

- This begins with an educated guess at how many lines of code are required to program a particular piece of functionality.
- E.g. to program a simple calculator of wages, taxes and other deductions might require 12000 lines of code. Assuming we are using COBOL. I have about 45 employees.
- It is a standalone, i.e. it does not integrate with any other system.

Organic: Standalone applications such as classical, non-web enabled word-processors.
Semidetached: In between.
Embedded: Integral to hardware and software systems.

Software project	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

$$Effort = a \times KLOC^{b}$$

$$Duration = c \times Effort^{d}$$

$$\therefore Duration = c \times (a \times KLOC^{b})^{d} = c \times a^{d} \times KLOC^{bd}$$

$$Effort = a \times kSLOC^{b}$$

If Application is organic, $a = 2.4$ $b = 1.05$

kSLOC – Thousands of Source Lines of Code.

My guess is 1000 Liner of Code = 1 kLOC.

$$Effort = 2.4 \times 1^{1.05} \approx 2.4 \text{ person-months}$$

My guess is 300 liner of Code 0.3 kLOC

$$Effort = 2.4 \times (0.3)^{1.05} \approx 0.7 \text{ person-months}$$

$$\text{Duration} = c \times \text{Effort}^{d} \qquad [\text{months}]$$

$$= 2.5 \times (0.7)^{0.38}$$

$$\approx 2.2 \quad \text{months}$$

$$\text{Person} = \frac{0.7}{2.2} = \frac{\text{person-months}}{\text{months}}$$

$$1 \quad \text{person}$$

$$Effort = a \times KLOC^{b}$$

KLOC - Thousands of Lines of code
K L O C

Since project is organic

$$Effort = 2.4 \times 12^{1.05} \quad [\text{person-months}]$$

$$= 2.4 \times 13.587523\overline{5}$$

$$= 32.610056\overline{41} \quad [\text{person-months}]$$

$$Duration = 2.5 \times 33.61005641^{0.38}$$

$$= 2.5 \times 3.7590109\overline{34}$$

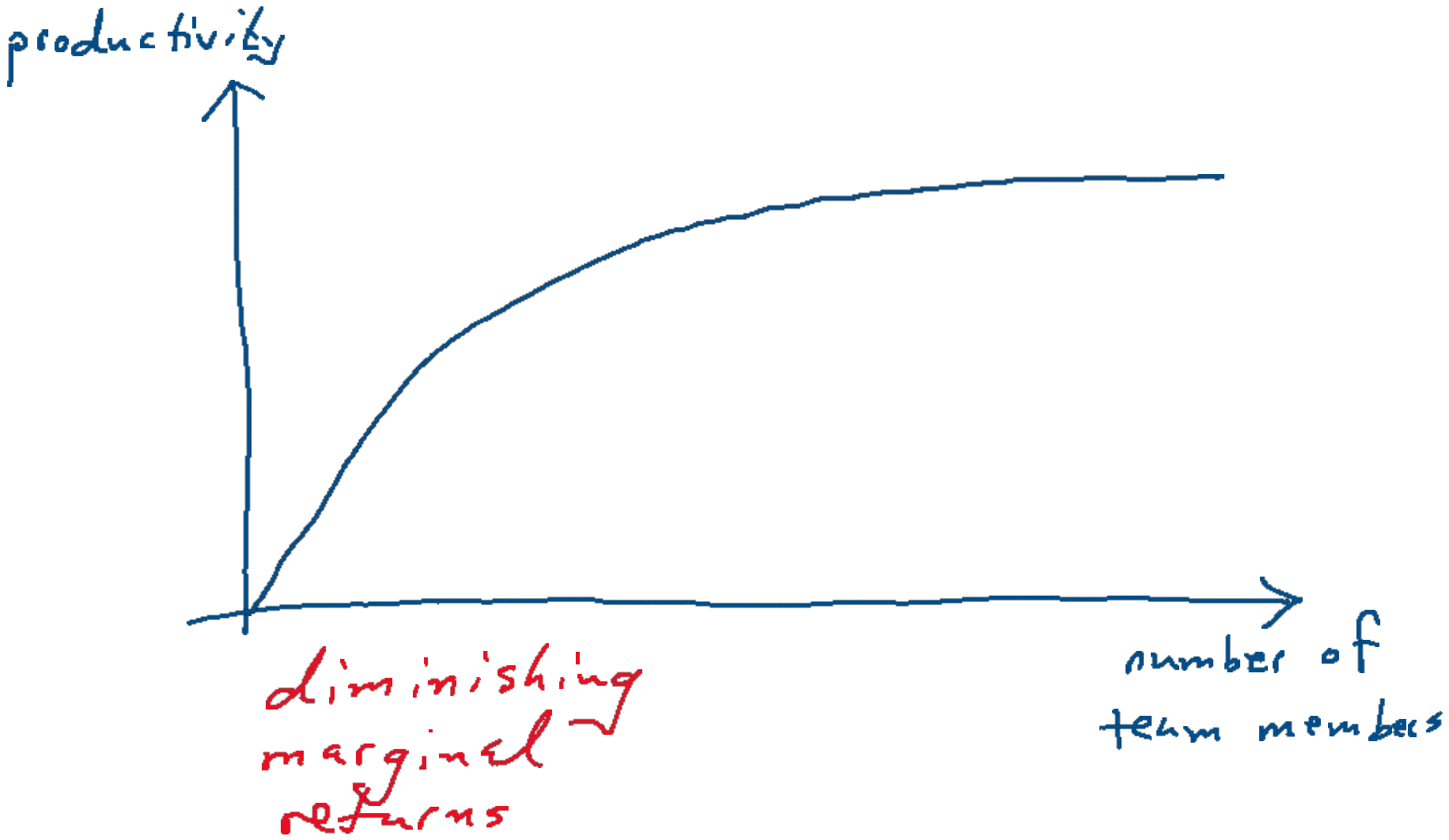
$$= 9.3972 \quad [\text{months}]$$

$$People = \frac{32,6}{9.397s^{-}} \quad \left[\frac{\text{person-months}}{\text{months}}\right]$$

$$\simeq 3.469$$

$$\simeq 4 \text{ people}$$

The mythical person-month



- Difficult task
- Simplest methods based on source lines of code (LOC) often expressed as thousands of lines of code (KLOC).
- LOC seems to be of less importance as productivity tools improve and languages become more productive.
- While estimation methods based on LOC have noted flaws, significantly better methods have not been widely reported.

Lines of code

- Basis: The amount of time required to develop a set of code is a function of its size.
- Begins with gathering historical records of experiences of past projects. The historical data are the basis for identifying the relationship between key measures of importance , e.g.:
  - person-months of effort
  - money expended
- Other factors include:
  - Pages of documentation generated
  - Errors encountered
  - System defects
  - People assigned

Lines of code

- For a new project an estimate of the number of lines is made. E.g. 10000 lines.

Effort	**1.606 x 10000 LOC = 16 person-months**
Budget	15.673 x 10000 LOC = R157 000
Documentation	58.122 x 10000 LOC = 581 pages
Errors	9.784 x 10000 LOC = 98
Defects	2.531 x 10000 LOC = 25
People	0.195 x 10000 LOC = 2 people

Lines of code

- The method is rough but provides a simple to implement approach.

Function Point Analysis

- Albrecht's function point (FP) analysis method supplements project size bases of estimation. The approach aims to provide a consistent meaningful measure with rules that are easy to apply.
- Can be used to estimate cost and time based on requirements specifications, and is independent of the technology used.
- It has been strongly criticized for questionable weightings, alternate counting methods and underweighting of large systems. However some consider it to be more useful than the LOC method.

Function Point Analysis

- Works in a manner similar to LOC, except the basis for estimation is function points (reflecting in essence work to be done). The original presentation of the method involved counts of the number of activities in five categories:
  1. User inputs
  2. User outputs
  3. User inquiries
  4. Files accessed
  5. External interfaces
- The number of functions are counted by complexity level for each factor and multiplied as in the following table.

Function Point Analysis

	Complexity Weighting			
Measurement parameter	Low or simple	Average or medium	High or very complex	Product
Number of user inputs	4 x 3 +	1 x 4 +	3 x 6 + =	12 + 4 + 18
Number of user outputs	6 x 4 +	0 x 5 +	0 x 7 + =	6 x 4
Number of user inquiries	0 x 3 +	5 x 4 +	0 x 6 + =	5 x 4
Number of files	10 x 7 +	0 x 10 +	0 x 15 + =	10 x 7
Number of external interfaces	0 x 5 +	15 x 7 +	0 x 10 + =	15 x 7
			Count Total	253

34

24

20

70

105

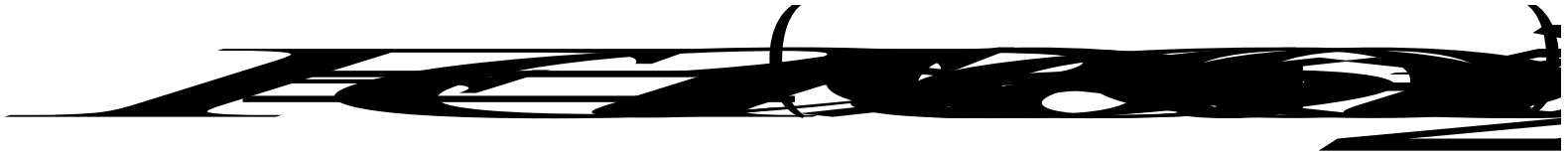
Cost Driver Table for Function Points Model

F1	Does the system require reliable backup and recovery?		5
F2	Are data communications required?		5
F3	Are there distributed processing functions?		3
F4	Is performance critical?		5
F5	Will the system run in an existing, heavily utilized operational environment?		0
F6	Does the system require online data entry?		5
F7	Does the online data entry require the input transaction on multiple screens or operations?		0
F8	Are the master files updated online?		0
F9	Are the inputs, outputs, files or inquiries complex?		0
F10	Is the internal processing complex?		2
F11	Is the code designed to be reusable?		1
F12	Are conversion and installation included in the design?		0
F13	Is the system designed for multiple installations in different organizations?		0
F14	Is the application designed to facilitate change and ease of use by the user?		0
	$\sum F_i$	Total	26

Function Point Analysis

- A 0 to 5 scale is used for these 14 factors:
  - 0 no effect
  - 1 incidental
  - 2 moderate
  - 3 average
  - 4 significant
  - 5 essential effect
- Function points are calculated using the following formula:

$$FP = Count\ total \times \left(0.65 + 0.01 \times \sum F_i\right)$$



$$\text{Function Points} = 253 \times (0,65 + 0,01 \times 26)$$

$$= 253 \times (0,65 + 0,26)$$

$$= 253 \times 0,91$$

$$= 230,23$$

Record of Performance on Past Projects

Average of past projects:	Effort	R(000)	pp. Doc.	Errors
LOC 12500 FP 600	12 months	521	300	45

FP = 600 => 12500 Lines of Code

12 Months to complete

R 521 000

300

45 errors hidden in our code

Use direction proportion to make our estimates.

$$\frac{FP}{LOC} = \frac{600}{12500} = \frac{230,23}{x}$$

$$\frac{LOC}{FP} = \frac{12500}{600} = \frac{x}{230,23}$$

$$\frac{x}{230,23} = \frac{12500}{600}$$

$$\frac{x}{230,23} \times 230,23 = \frac{12500}{600} \times 230,23$$

$$x = \frac{12500}{600} \times 230,23$$

$$x \approx 4797 \text{ Lines of Code}$$

Duration / Effort

$$\frac{x}{230,23} = \frac{12}{600} = \frac{\text{Effort}}{FP}$$

$$x = \frac{12 \times 230,23}{600} \text{ months}$$

$$\approx 4,6 \text{ months}$$

Function Point Estimation

- Use of the *FP* value is based on averages of past projects (regression?)
- Function Points: Calculate *FP*, multiply averages by *FP*.
- I.e. Average per *FP* from past projects, Effort, Budget, Documentation, Errors, Defects, People

- Many other software estimation models have been developed. However, many are proprietary models and cannot be readily compared and contrasted in terms of the model structure. Theory or experimentation determines the functional form of these models.

(Constructive Costing Model) COCOMO

- Now considered to be obsolete
- The Basic COCOMO computes software development effort based on program size.
- An intermediate model also considers a set of cost drivers reflecting specific product, hardware, personnel, and project characteristics.
- For relatively small, simple software projects built by small teams with good experience, COCOMO formulas for person-months of effort and development time in chronological months are as follows:

$$Person\ months = 2.4 \times KLOC^{1.05} = E\ \ for\ effort$$

$$Duration\ (months) = 2.5 \times E^{0.38}$$

COCOMO ‘81 Intermediate Model

- Constructive Costing Model
- Where KLOC means thousand lines of code.
- There are variations of COCOMO in current use.
- It is possible for organizations to generate their own using regression on productivity data.
- While the COCOMO approach is not perfectly accurate in many conditions, it nevertheless provides a useful starting basis for projects oriented around work that can be described in terms of lines of code generated.

- For software projects of intermediate size and complexity, built by teams with mixed experience and facing more rigid than average requirements the formulas are modified as follows: $Person\ months = 3.0 \times KLOC^{1.12} = E\ for\ effort$

$$Duration\ (months) = 2.5 \times E^{0.35}$$

- Finally, for a software project built under rigid conditions, the formulas are as follows:

$$Person\ months = 3.6 \times KLOC^{1.2} = E\ for\ effort$$

$$Duration\ (months) = 2.5 \times E^{0.32}$$

COCOMO Cost Drivers

COST DRIVER	DESCRIPTION
RELY	Required software reliability
DATA	Database size
CPLX	Product complexity
TIME	Execution time constraints
STOR	Main storage constraints
VIRT	Virtual machine volatility - degree to which the operating system changes
TURN	Computer turn around time
ACAP	Analyst capability
AEXP	Application experience
PCAP	Programmer capability
VEXP	Virtual machine (i.e. operating system) experience
LEXP	Programming language experience
MODP	Use of modern programming practices
TOOL	Use of software tools
SCED	Required development schedule

How to use the Intermediate Cocomo ‘81 model?

- We imagine we are developing an application that we believe needs 12000 lines of code to complete.
- It is an organic project.
- Our developers are very highly capable. [PCAP]
- Our database is very large. [DATA]

$$Effort = a \times KLOC^{b} \times \text{product of cost drivers}$$

$$= a \times KLOC^{b} \times DATA \times PCAP$$

$$= 32,6 \times 1.16 \times 0.70 \approx 26.5$$

COCOMO Cost Driver Values

COCOMO - COST DRIVERS

	COST DRIVER	RATING V.LOW	LOW	NOMINAL	HIGH	V.HIGH	EX. HIGH
(PRODUCT)	RELY	0.75	0.88		1.15	1.40	.
..	DATA	.	0.94	1.00	1.08	.	.
..	CPLX	.	0.85	1.00	1.15	1.30	1.65
(COMPUTER)	TIME	.	.		1.11	1.30	1.66
..	STOR	.	.		1.06	1.21	1.56
..	VIRT	.	0.87		1.15	1.30	.
..	TURN	.	0.87		1.07	1.15	.
(PERSONNEL)	ACAP	1.46	1.19		0.86	0.71	.
..	AEXP	1.29	1.13		0.91	0.82	.
..	PCAP	1.42	1.17	1.00	0.86		.
..	VEXP	1.21	1.10		0.90		.
..	LEXP	1.14	1.07		0.95	.	.
(PROJECT)	MODP	1.24	1.10		0.91	0.82	.
..	TOOL	1.24	1.10		0.91	0.83	.
..	SCED	1.23	1.08		1.04	1.10	.

$$E = 3.0 \times KLOC^{1.12} \times product(cost\ drivers) = E\ \ for\ effort$$

Effort = a * kLOC^b *

Basic COCOMO [8]

$$Effort = a \times (KLOC)^{b} \quad [\text{person-months}]$$

$$Duration = c \times Effort^{d} \quad [\text{months}]$$

$$People = \frac{Effort}{Duration} \quad \left[\frac{\text{person-months}}{\text{month}}\right]$$

Intermediate COCOMO '81

$$Effort = a \times (KLOC)^{b} \times \text{product of Cost Dr. UP}$$

Multiply your cost drivers. All 14 of them

$$Effort = a \times (KLOC)^{b} \times 0.70$$

$$Duration = c \times Effort^{d} \quad [\text{months}]$$

$$People = \frac{Effort}{Duration} \quad \left[\frac{\text{person-months}}{\text{month}}\right]$$

In Summary

Organic: Standalone applications such as classical, non-web enabled word-processors.
Semidetached: In between.
Embedded: Integral to hardware and software systems.

Software project	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

$$Effort = a \times KLOC^{b}$$

$$Duration = c \times Effort^{d}$$

$$\therefore Duration = c \times (a \times KLOC^{b})^{d} = c \times a^{d} \times KLOC^{bd}$$

- The difference between the basic Cocomo model and the intermediate is in the use of cost drivers
- The basic model does not use cost drivers
- The intermediate model uses up to 15 cost drivers.

- So, one way to estimate Cost and Duration very early in a project
  1. Use function point method to estimate lines of code.
  2. Use Boehm's COCOMO formulas to estimate labour required.
  3. User and labour estimate and Boehm's formulas to estimate duration.

COCOMO Conclusion

- Boehm’s duration formula is independent of the number of people put on the job. It depends only on the size of the job. It assumes that the project will have roughly an appropriate number of people available to it at any given time.
- The model has been tested extensively and is widely recognized, and has been refined over time. However, a great deal of practice is required to use it effectively. It has probably best used along with an independent method and a healthy dose of common sense.

QUESTION 5 **[12]**

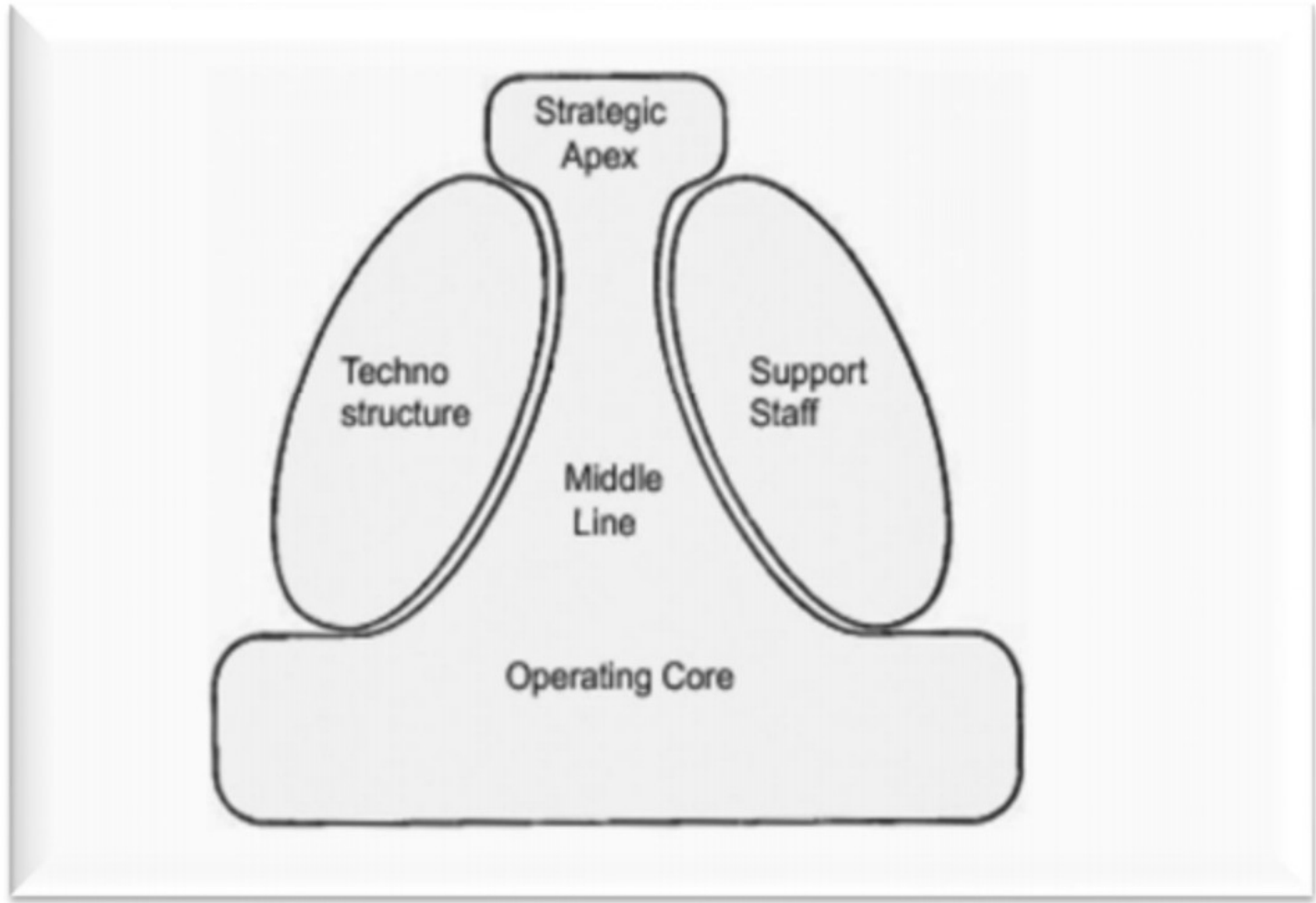
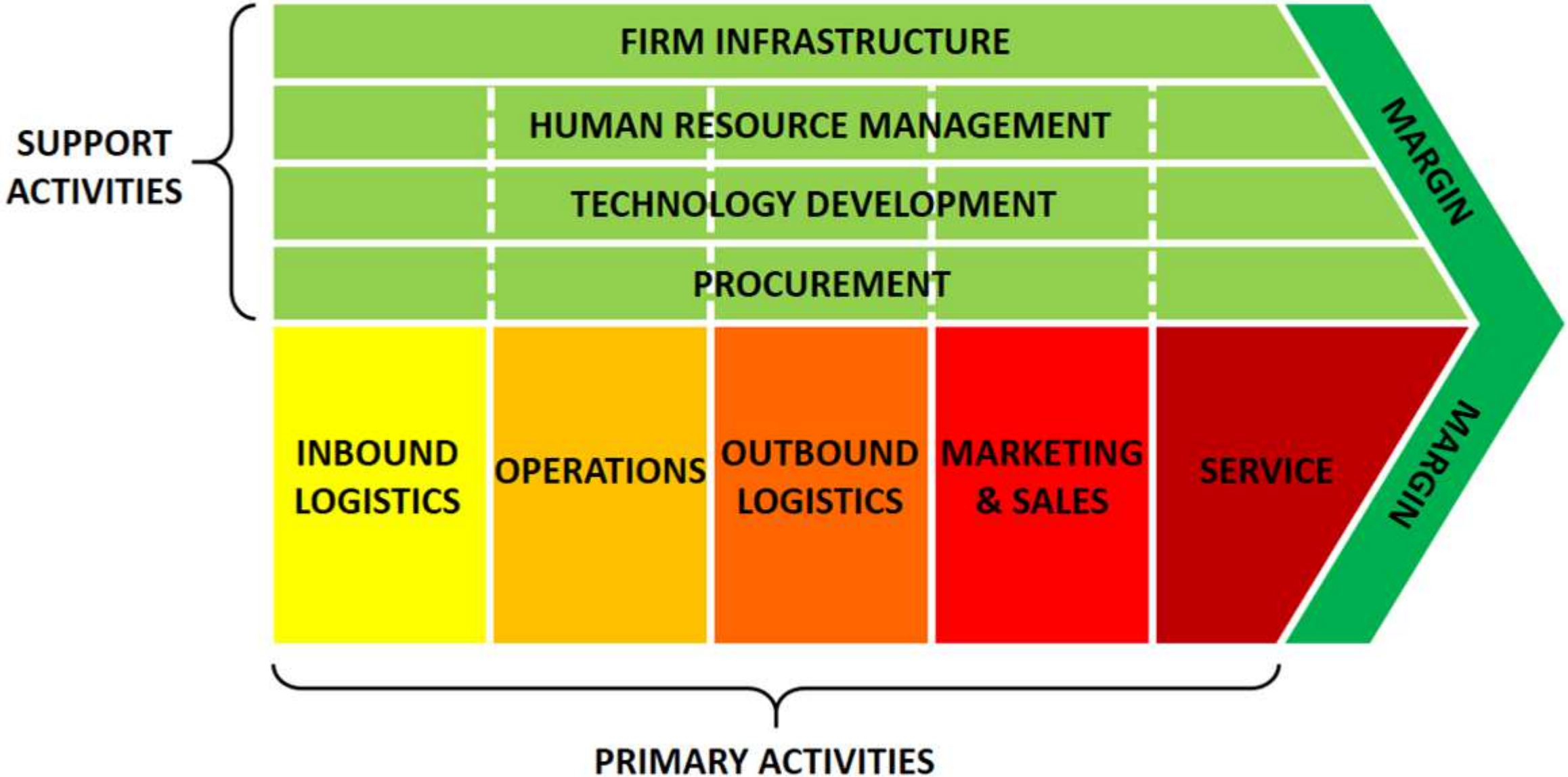


Figure 1: Structure of an organisation (Source: Mintzberg, 1979: 147)

Most organisations comprise five clearly definable parts, and each has a specific function. The parts are indicated as the strategic apex, middle line, operating core, support staff, and techno structure. Describe these parts in detail.


SUPPORT ACTIVITIES
FIRM INFRASTRUCTURE
HUMAN RESOURCE MANAGEMENT
TECHNOLOGY DEVELOPMENT
PROCUREMENT
INBOUND LOGISTICS
OPERATIONS
OUTBOUND LOGISTICS
MARKETING & SALES
SERVICE
MARGIN
MARGIN
PRIMARY ACTIVITIES

- Why are we doing a project?
  - Strategic
  - Tactical
- Work Breakdown Structure (WBS)
  - Tactical and a bit of strategic